

Faster Code, Deeper Debt? A Multivocal Literature Review on Technical Debt and Its Early Signs in LLM-Assisted Software Development

RAMTIN EHSANI, Drexel University, USA

SHRIYA RAWAL, Drexel University, USA

YUANFANG CAI, Drexel University, USA

PREETHA CHATTERJEE, Drexel University, USA

With the rapid adoption of LLM-assisted coding, the need to manage the technical debt these systems introduce has become urgent. In this paper, we conduct a multivocal literature review of 104 sources (31 formal, 73 grey) to examine how LLM-assisted development contributes to technical debt and what strategies, metrics, and benchmarks exist to mitigate it. We find that LLMs often amplify traditional forms of technical debt, particularly code, design, and documentation debts, while also introducing new LLM-specific debts. Notably, we identify fast-integration debt, where rapidly generated code prioritizes speed over quality, triggering a domino effect that leads to governance debt and increased long-term maintenance costs. Additional emerging categories include prompt, ethical, data, and provenance debt, reflecting new challenges unique to LLM adoption. To address these, strategies suggested in the literature include human-in-the-loop frameworks, prompt engineering, and data quality alignment. In practice, tools such as SonarQube are commonly used to detect technical debt indicators, while research prototypes such as CodeSmellEval are emerging to assess how LLMs contribute to debts. However, no standardized benchmarks or LLM-specific metrics yet exist, leaving an important gap. Based on findings, we outline insights and future directions to ensure reliable integration of LLMs into software engineering workflows.

CCS Concepts: • **Software and its engineering** → **Software creation and management**; • **Computing methodologies** → **Intelligent agents**; • **General and reference** → **Surveys and overviews**.

Additional Key Words and Phrases: large language models, AI agents, code generation, technical debt

1 Introduction

Large language models (LLMs) are rapidly transforming how developers write and maintain code. These tools promise increased productivity and reduced development effort [207], yet their long-term implications, particularly regarding technical debt, remain insufficiently understood. Developers are increasingly relying on LLMs for quick fixes, boilerplate, and even entire modules, but the resulting code may suffer from inconsistency, poor testing, limited documentation, or reduced maintainability [165, 198, 200]. These potential risks warrant closer attention as LLMs become a regular part of software development practice, especially with the emerging vision of AI-native Software Engineering (SE 3.0), where human-AI collaborations become more integrated into the development workflows [89, 120].

Authors' Contact Information: Ramtin Ehsani, ramtin.ehsani@drexel.edu, Drexel University, Philadelphia, Pennsylvania, USA; Shriya Rawal, sr3728@drexel.edu, Drexel University, Philadelphia, Pennsylvania, USA; Yuanfang Cai, yc349@drexel.edu, Drexel University, Philadelphia, Pennsylvania, USA; Preetha Chatterjee, preetha.chatterjee@drexel.edu, Drexel University, Philadelphia, Pennsylvania, USA.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1557-7392/2026/6-ART

<https://doi.org/10.1145/3820165>

Emerging evidence points to a possible trade-off between short-term productivity gains and long-term software maintenance efforts. While several studies suggest that AI tools can boost developer productivity [2, 42, 151], their broader implications for long-term maintenance remain unclear. For example, a large-scale analysis by GitClear, which analyzed 211 million lines of code from major technology companies between 2020 and 2024, reported increased code duplication, short-term churn, and a sharp decline in code reuse [1]. For the first time, “copy/paste” style duplication outpaced refactored code reuse, signaling a shift toward practices that may accelerate the accumulation of technical debt [1]. Because these issues often do not cause immediate failures, they can be easily overlooked during developer reviews [124].

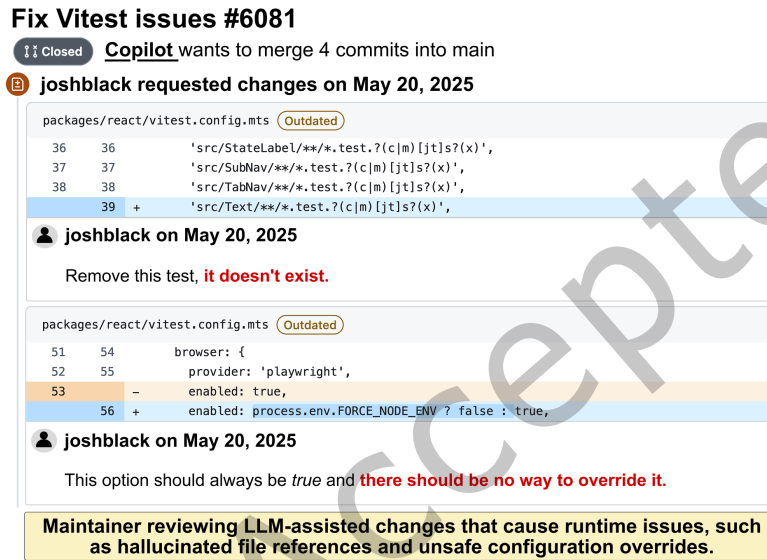


Fig. 1. Example of LLM-assisted code (by Copilot) submitted to project *primer/react* on GitHub (3.8k stars), highlighting new risks such as hallucinated file references and unsafe configuration overrides.

LLM-generated code introduces risks that differ in important ways from those of traditional human-written code. For instance, Figure 1 shows a pull request involving LLM-assisted code generation using GitHub Copilot¹. The generated code includes hallucinated file references and unsafe configuration overrides, highlighting new risks that are unique to LLM-assisted development workflows [124]. LLM-generated code can also be highly prompt-dependent. Small changes in prompts or model settings (e.g., higher temperature) can produce different implementations for the same functionality, making system behavior harder to reproduce [199]. Moreover, prompts and inputs to LLMs themselves become first-class development artifacts, introducing new challenges in how they are preserved, versioned, and maintained over time [193]. Unlike human-written code, which reflects deliberate design decisions and traceable authorship, LLM-generated code is often produced non-deterministically. This can lead developers to integrate code whose logic or assumptions are not fully understood, increasing the cognitive effort required to evaluate, maintain, and eventually repay the technical debts it may introduce [187]. These potential risks and their early signals must be understood more thoroughly to support the reliable integration of LLMs into software engineering workflows.

¹<https://github.com/primer/react/pull/6081>

While prior work has examined correctness and quality issues in LLM-generated code [101, 123], these perspectives primarily capture local or task-specific software quality concerns, but do not fully explore how such issues could accumulate into long-term technical debt. This distinction is critical in the context of LLM-assisted development, where the literature is still emerging and explicit discussions of technical debt remain limited. Thus, we conduct a systematic Multivocal Literature Review (MLR) on technical debt and its early signals in LLM-Assisted software development to support a holistic understanding of long-term trade-offs and compounding costs across the integration of code and governance of AI-based software systems.

1.1 Research Objective

To provide a structured overview of this emerging topic, we conduct a Multivocal Literature Review (MLR) that integrates evidence from both practitioner grey literature (e.g., blogs, online forums, industry reports) and academic publications. This dual perspective allows us to capture not only the practical concerns voiced by developers in real-world settings but also the analytical and methodological insights provided by research studies. Our objective is to comprehensively examine the forms of technical debt and their associated signs with LLMs, the risks and challenges they introduce into software projects, and the strategies, tools, and metrics currently available or still missing for managing them. To guide this investigation, we address the following research questions:

- **RQ1:** *What forms of technical debt are introduced by integrating LLM-generated code into software projects?*
- **RQ2:** *What strategies or guidelines exist to mitigate technical debt introduced by LLMs?*
- **RQ3:** *What tools or techniques currently exist to help practitioners detect or measure technical debt in LLM-generated code?*
- **RQ4:** *Are there benchmarks or datasets available to evaluate the technical debt of LLM-generated code?*
- **RQ5:** *Do existing technical debt metrics sufficiently capture the debt introduced by LLMs? If not, what new metrics or dimensions are being suggested?*

1.2 Contributions

Our review highlights the complex landscape of integrating LLM-generated code into software systems and its implications for technical debt. Through a systematic examination of the existing literature, we analyze how 13 types of technical debt (as described in Alves et al.'s taxonomy [16]) manifest in the context of LLMs, identify novel forms of debt unique to the usage of LLMs, and map the strategies, tools, and metrics discussed for their mitigation. In doing so, we also surface research gaps and point toward future directions that the community should consider. The specific contributions of this literature review are as follows:

- We systematically analyze 31 formal and 73 grey sources, providing the first multivocal perspective on technical debt in LLM-generated code.
- We examine how different types of traditional technical debt manifest in LLM-generated code and show that traditional debts such as *code*, *design*, and *documentation* debt are the most frequently observed in LLM-generated code, often arising from issues such as poor testing and lack of governance.
- We highlight emerging, LLM-specific forms of debt (e.g., prompt debt, provenance debt) that go beyond the traditional definitions of technical debt.
- We synthesize reported strategies (e.g., human-in-the-loop frameworks) and tools (e.g., SonarQube, Snyk) aimed at mitigating technical debt in LLM-augmented software development.
- We highlight key gaps, including the lack of standardized benchmarks, the absence of LLM-specific metrics (e.g., semantic consistency of LLM outputs), and the limited availability and scope of tools for detecting technical debt induced by LLMs, thereby outlining concrete opportunities for future research.

All of our assessments, along with additional materials, are available in our supplementary materials ².

1.3 Organization of the Paper

Section 2 provides a background on technical debt studies in software engineering. Section 3.1 expands on the study's objectives and research questions. Section 3.2 details the search strategy and source selection process, followed by Section 3.3, which describes our data extraction and synthesis procedures. Section 4 presents the findings of our multivocal literature review by presenting evidence extracted directly from the analyzed sources for each research question, and Section 5 builds on these results by synthesizing the findings, discussing their implications for researchers and practitioners, identifying open research gaps, and outlining directions for future work. Section 6 provides threats to validity, and finally, section 7 presents the conclusions.

2 Related Work

2.1 TD in Traditional Software Systems

Technical debt in software has been widely studied through numerous secondary studies and mapping studies. Prior research has primarily focused on traditional software development settings, examining debt types such as code, design, and architectural debt, and investigating dimensions such as measurement, prioritization, tool support, and management strategies.

Besker et al. [35] synthesized knowledge on architectural debt and proposed a descriptive model to classify and analyze architectural debt, while Sousa et al. [183] and Koulla Moulla et al. [140] reviewed methods and tools for the identification and measurement of architectural debt. Other works studies examined approaches for quantifying code, design, and architecture technical debt [153], techniques for prioritizing technical debt [11, 118], and requirements-related debt [133, 152]. Several studies also investigated technical debt in specific contexts, such as agile development [32], microservices [201], continuous engineering [126], and systems engineering [113]. Collectively, these reviews have produced taxonomies, roadmaps, and decision-making models that strengthen understanding of technical debt in traditional software projects.

Over the past decade, significant advances have been made in detecting design debt at the architectural and design levels. Prominent tools include Designite [63], DV8 [21, 138, 206], Structure101 [189] (recently acquired by SonarQube), Archinaut [46], and Arcan [79, 80]. CodeScene [53] represents a novel approach by identifying technical debt based on revision history analysis. In addition, a wide range of other tools detect technical debt in various forms, including SonarGraph [203], CodeInspector [100], CodeMRI [141], SQuORE [184], SymphonyInsight [191], inFusion [149], JDeodorant [149], PMD [154], Checkstyle [47], and JSPiRiT [105]. Fontana et al. [78] were the first to conduct a comparative evaluation of different code smell detection tools. They analyzed six versions of a software system using four tools: Checkstyle, inFusion, JDeodorant, and PMD. Fernandes et al. [75] later presented a systematic literature review covering 29 tools, each evaluated individually for two specific smells: Large Class and Long Method. Their study involved two software systems and compared the tools using three metrics: agreement, recall, and precision. Thanis et al. [149] further assessed the accuracy of these four tools in detecting three code smells: God Class, God Method, and Feature Envy. Code smells are widely recognized as indicators of potential technical debt, often signaling deeper issues that may accumulate over time if left unaddressed. Therefore, research on code smell detection is closely related to technical debt management and provides important context for understanding how early quality issues can evolve into long-term liabilities. More recent research includes the work of Avgeriou et al. [26], which compared various technical debt detection tools, and the study by Biazotto et al. [36] that examined the state of the art in automated technical debt detection. Similarly, Amanatidis et al. [17] and Lefever [117] investigated the accuracy and agreement among multiple technical debt detection tools, revealing a persistent lack of consensus on what constitutes real technical debt. To

²<https://figshare.com/s/6d94133d27a880729d6b>

the best of our knowledge, most of these techniques and tools, particularly those operating at the architecture or design level, have not yet been applied to software systems generated by large language models (LLMs).

2.2 TD in AI/ML Systems

More recently, studies have begun to explore technical debt in AI and ML systems, primarily focusing on the development and operation of the models themselves rather than on the downstream integration of their outputs in software projects. Bogner et al. [40] identified new debt types in AI-based systems, such as data, model, configuration, and ethics debt, showing how these differ from conventional forms by arising directly from training data quality, model lifecycle management, and ethical risks in deployment. Albuquerque et al. [10] reviewed the use of intelligent techniques (ML, AI, data mining) for managing technical debt, highlighting how AI itself is increasingly being applied as a tool for debt detection and remediation. Saeeda et al. [168] broaden the scope by examining non-technical debts (process, people, social, organizational), emphasizing that AI adoption not only creates new technical challenges but also reshapes collaboration, organizational practices, and cultural norms around debt management. Moreschini et al. [139] examine the evolution of technical debt from DevOps to the most recent generative AI stage, finding that Machine Learning Operations (MLOps) practices are increasingly recognized as essential for managing data-related, infrastructure and pipeline-related technical debt, especially in relation to dynamic data dependencies and model retraining. They also emphasize that AI-generated artifacts and automated pipelines introduce new governance and maintainability challenges. Together, these signal a gradual shift in the literature from studying technical debt purely in traditional software systems toward capturing the broader, system-level, and socio-technical debts that emerge in AI and LLM-enhanced development.

Despite this growing body of work on technical debt, most studies concentrate on classical technical debt types or ML-specific issues, such as model versioning, concerning the model deployment aspect of organizations that build and maintain AI systems. Very few studies explicitly examine the role of LLMs or the debts that arise when developers use them in workflows, or when LLM outputs are integrated into software projects. This review is among the first to bridge that gap by providing a multivocal and up-to-date perspective, drawing on both academic and practitioner sources, focusing on technical debt that emerges when LLMs and their outputs are integrated into software development. We show how LLMs not only amplify traditional forms of debt but also give rise to entirely new categories that demand fresh attention in software engineering research and practice, and what strategies we can use to tackle these challenges in the future.

3 Methodology

Our MLR process is summarized in Figure 2. The process begins with *MLR Planning* phase, where we define the study goals and research questions. Next, in the *Source Selection* phase, we first search formal literature and grey literature using a specific search string on a variety of data sources. We then apply inclusion and exclusion criteria through independent voting and discussion, resulting in a final pool of selected sources. Next, we move on to the *Classification Scheme/Map* phase, where we conduct attribute identification and data extraction, capturing both metadata (e.g., authors, year, source type) and study-specific fields (e.g., debt types, strategies, tools, metrics). These attributes are refined through iterative coding and reliability checks. Finally, in the *Systematic Mapping, Synthesis and Review* phase, we organize and compare findings from grey and formal sources, creating a classification map of debt types, strategies, tools, and metrics, and synthesizing results to answer the research questions. To conduct our literature review, we follow the widely adopted MLR guidelines for SE proposed by Garousi et al. [82], while also adhering to the SE Guidelines for Reporting Secondary Studies (SEGRESS) by Kitchenham et al. [111] built on PRISMA. In addition, we draw inspiration from recent thematically related review studies [28, 83] in terms of search strategy design, grey literature selection, quality assessment, and synthesis of practitioner and academic perspectives.

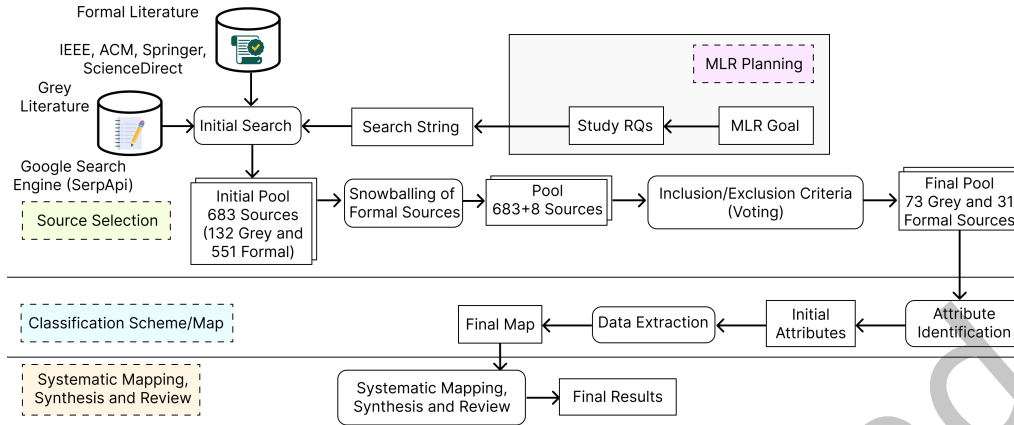


Fig. 2. An Overview of our Literature Review Process.

3.1 Goal and Research Questions

The goal of this study is to systematically map, review, and synthesize the current state of both academic research and practitioner perspectives on technical debt in LLM-assisted software development. We aim to uncover emerging trends, classify reported issues, and identify directions for future research, viewed from the standpoint of both researchers and practitioners. Technical debt has long been a critical topic in software engineering, even before the rise of AI and LLM-based tools [121, 195]. As these tools become increasingly integrated into software development workflows, they offer benefits such as automation and reduced manual effort [95]. It is also important to examine their potential long-term effects, particularly in the form of hidden or unintentional technical debt. Our study was motivated by recurring themes in blogs and interviews suggesting that “*LLMs are enabling the rapid creation of technical debt at scale*” [57]. Technical debt often takes time to surface within a codebase [72, 121, 195]. Given that LLM-assisted code generation has gained more traction since 2021 [102], it is important to understand the effects of increased adoption of these models on long-term software maintainability.

One of the key aspects of an MLR is the justification for including grey literature. Garousi et al. [82] propose a set of seven guiding questions to assess whether a grey source should be included in an MLR (see Table 1). According to this framework, one “yes” answer to any of the questions supports its inclusion. In our case, while there is an emerging body of formal research on technical debt in the context of LLMs, it takes time for such work to reach publication. Grey literature provides timely, real-world perspectives that complement the gaps in academic studies. Given the increasing adoption of LLMs in both industry and research and their growing influence on software engineering practices [95], it is critical to conduct a comprehensive multivocal review. The inclusion of grey literature in our study is therefore essential to ensure that emerging practitioner concerns are represented and can inform future academic work.

Based on the goals and motivations outlined above, we classify and analyze sources from both grey and formal literature to understand the discourse on technical debt from LLM-generated code and how widely it is being discussed. We investigate the following research questions to guide our study, each formulated to be concrete, answerable, and aligned with the dual perspectives of both researchers and practitioners:

- **RQ1:** *What forms of technical debt are introduced by integrating LLM-generated code into software projects?* To categorize the types of technical debt, we apply the taxonomy by Alves et al. [16], which includes 13 types: *Architecture Debt, Build Debt, Code Debt, Defect Debt, Design Debt, Documentation Debt, Infrastructure Debt, People Debt, Process Debt, Requirement Debt, Service Debt, Test Automation Debt, and Test Debt* (the definitions

Table 1. Questions to Decide whether to Include the Grey Literature in our Study.

#	Question	Possible Answers	MLR-TechDebt
1	Is the subject “complex” and not solvable by considering only the formal literature?	Yes/No	Yes
2	Is there a lack of volume or quality of evidence, or a lack of consensus on outcome measurement in the formal literature?	Yes/No	No
3	Is the contextual information important to the subject under study?	Yes/No	Yes
4	Is it the goal to validate or corroborate scientific outcomes with practical experiences?	Yes/No	Yes
5	Is it the goal to challenge assumptions or falsify results from practice using academic research or vice versa?	Yes/No	No
6	Would a synthesis of insights and evidence from the industrial and academic community be useful to one or even both communities?	Yes/No	Yes
7	Is there a large volume of practitioner sources indicating high practitioner interest in a topic?	Yes/No	Yes

Note: One or more “yes” responses suggest inclusion of grey source.

are provided in Table 8). This helps identify which aspects of software are most impacted by LLM-generated code. We also examine to identify new or emerging forms of debt that may be unique to LLM usage.

- **RQ2:** *What strategies or guidelines exist to mitigate technical debt introduced by LLMs?* We examine whether the sources provide practical suggestions, heuristics, or formal strategies for minimizing or addressing the technical debt caused by LLM-assisted code generation.
- **RQ3:** *What tools or techniques currently exist to help practitioners detect or measure technical debt in LLM-generated code?* This question explores existing tools, such as linters, static analysis tools, or AI-specific detectors that the literature uses or recommends to identify and measure technical debt and its early signs in LLM-generated code.
- **RQ4:** *Are there benchmarks or datasets available to evaluate the technical debt of LLM-generated code?* We investigate whether any datasets, curated repositories, or challenge sets are available that can support systematic evaluation of LLM-induced technical debt.
- **RQ5:** *Are there existing technical debt metrics that sufficiently capture the debt introduced by LLMs? If not, what new metrics or dimensions are being suggested?* We investigate if there are standard metrics (e.g., those used by tools like SonarQube) that are adequate to identify technical debts introduced by LLMs, or if the literature proposes new metrics or extensions to better capture the characteristics of LLM-generated code.

3.2 Search and Source Selection

Our search was conducted in January 2026, and therefore, our study includes all sources available up to January 2026. To determine the most appropriate search query, we conducted multiple rounds of exploratory search refinement and collaborative discussions to ensure comprehensive coverage of both grey and formal literature. In the initial phase, we examined individual keywords and keyword combinations (e.g., “technical debt” AND “LLM”, or “AI-generated code”) to assess the relevance and comprehensiveness of the returned results. This step allowed us to observe which terms were more effective in surfacing studies directly related to our research questions and which terms introduced excessive noise. After these iterative exploratory investigations, we aggregated the terms into a query string. Through additional iterations, we refined the logical structure of the query, combining related

Table 2. Quality Assessment Checklist of Grey Literature Sources.

Criteria	Questions
Authority	Is the author associated with a reputable organization? Is the publishing organization reputable? Has the author published other work in the field? Does the author have expertise in the area?
Methodology	Does the source have a clearly stated aim or brief? Does the source have a clearly stated methodology? Is the source supported by authoritative, documented references or credible sources? Does it specify how technical debt (maintainability, code quality, code smell, refactoring code written by AI) was identified or measured? Does it reference any code, data, or models shared? Are any assumptions/bias clearly stated?
Objectivity	Are the conclusions supported by data? Is the statement transparent about potential biases?
Date	Does the item have a clearly stated date? How current is the piece? Does it reference recent models or frameworks?
Significance	Does it enrich or add something new to the research? Does it strengthen or refute a current position (takeaways)? Does it directly address technical debt? Are failure modes, scaling issues, or long-term maintenance challenges described? Are claims supported with experiments, metrics, or real-world cases?
Impact	Does it offer actionable insights or tools?
Outlet Type	1st tier source (High outlet): government report, white papers, company/technical blogs and reports from labs 2nd tier source (Moderate outlet): conference slides, news articles, developer forums and platforms 3rd tier source (Low outlet): personal blogs, tweets, posts

terms under grouped operators. The final version of the query string, presented below, consistently yielded the most relevant and comprehensive set of results across databases and search engines.

("technical debt" OR "code debt" OR "design debt" OR "documentation debt" OR "infrastructure debt" OR "architecture debt" OR "test debt" OR "defect debt" OR "defect debt" OR "build debt" OR "requirement debt" OR "service debt" OR "test automation debt" OR "process debt") AND ("large language models" OR "AI-generated code" OR "LLMs" OR "LLM" OR "ChatGPT" OR "Copilot" OR "AI code" OR "AI assistance")

We initially only used the term "technical debt" in our search string, but this risked missing studies that discuss specific debt types without explicitly using the umbrella term. To mitigate this limitation, we expanded the search string to include the 13 well-established technical debt categories defined by Alves et al. [16], which allowed us to capture a broader and more relevant set of sources.

3.2.1 Grey Literature. To identify relevant grey literature sources discussing technical debt in the context of LLMs, we used Google as our primary search engine. Prior multivocal literature reviews in software engineering

have shown that Google is effective in retrieving both highly relevant and credible grey literature, due to its robust PageRank algorithm [82, 83]. To this end, we leveraged SerpAPI³, a Google Search API that returns structured JSON results, allowing us to collect and analyze search results efficiently. This tool also removes all the cookies and potential bias in the retrieved links, which makes it a better choice over direct engine search by individuals.

Our search query returned approximately 375,000 results. However, based on our observations, the most relevant and high-quality sources were concentrated within the first 8-10 pages of search results. Beyond this range, the results became increasingly noisy, often consisting of unrelated content and broad topics of LLMs and AI, tweets, or promotional videos. Following the guidelines by Garousi et al. [82], which recommend capping grey literature searches once relevance saturates, we limited the sources to the first 10 pages. After removing duplicate entries and filtering out formal academic papers that appeared in the Google search results, we identified 132 unique grey literature sources relevant to our study.

To assess the credibility and quality of these sources, we followed the evaluation framework proposed by Garousi et al. [82], adapting it to our focus on technical debt in LLM-generated code. As shown in Table 2, our quality assessment includes 22 criteria across 7 broad categories (e.g., *Authority*, *Methodology*), covering aspects such as source authority and methodological clarity. The 132 sources were divided between two authors, who independently assessed the quality of each source. Specifically, for each source, we applied all 22 criteria: 1 (meets the criterion) or 0 (does not meet the criterion). For the *Outlet Type* criterion, a partial score of 0.5 was allowed to account for second-tier platforms. After the initial assessments, the two authors cross-checked the scored sources to ensure consistency in interpretation and application of the criteria. Any discrepancies were discussed and resolved through iterative consensus-based review, resulting in a final agreed-upon assessment for each source. The reported agreement of 1.0 (100%) therefore reflects consensus after reconciliation, rather than an initial inter-rater reliability statistic. To further strengthen the rigor of the process, a third author subsequently reviewed all annotations for consistency.

The maximum possible score per source was 22. Following Garousi et al.'s [82] recommendation, any source scoring above 11 (i.e., at least 50%) was deemed to have sufficient quality for inclusion. As a result, we finalized a pool of 73 credible grey literature sources to include in our study. Figure 3 shows the distribution of quality assessment scores for these grey literature sources. The scores are centered around a mean of 14.4 (SD = 2.52), with a 75th percentile of 15.5, indicating that the majority of included sources meet a moderate to high quality threshold.

To further assess the quality and reliability of our 73 sources, we follow the “shades of grey” classification model [82], to systematically categorize them based on the outlet type, expertise of the author or organization, and degree of editorial control. This model maps sources along two dimensions of expertise and outlet control to determine their tier (1st, 2nd, or 3rd), helping assess the quality and reliability of sources. We group our sources into four categories: *Company Blogs*, *Community and Developer Platforms*, *Personal Blogs*, and *Media Outlets*.

Company Blogs make the majority of our sources, with 38 sources. These blogs are published by established software engineering companies, AI startups, or enterprise tool providers such as IBM, Sonar, and Salesforce. Given the clear authorship and organizational control, these sources fall into the 1st tier of grey literature, characterized by high domain expertise and moderate-to-strong editorial oversight. *Community and Developer Platforms*, such as Medium, Hacker News, and Reddit, contribute 18 sources. These platforms offer practitioner-driven content and valuable real-world perspectives. However, they often vary in terms of authorship transparency and editorial control, with some content being anonymous or minimally moderated. As a result, these sources fall into the 2nd tier, where moderate expertise and limited outlet control are typical. *Media Outlets*, including sources like NewsStack and Visual Studio Magazine, account for 14 sources. These are typically tech journalism websites, industry news platforms, or blogs affiliated with academic and professional organizations. They are generally

³<https://serpapi.com/>

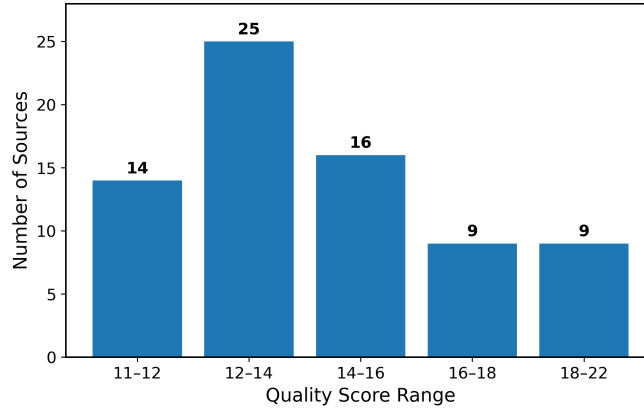


Fig. 3. Distribution of Quality Assessment Scores for Selected Grey Literature Sources.

Table 3. Grey Literature Sources by Tier and Type.

Tier	Type	Sources	Count
1	Company Blogs	[5, 8, 12, 19, 23, 27, 29, 38, 39, 41, 45, 49, 52, 56, 67, 81, 84, 86, 91, 94, 97, 98, 106, 115, 142, 143, 148, 155, 160, 162, 166, 173, 174, 178, 179, 181, 188, 194]	38
2	Developer Platforms	[4, 7, 20, 34, 48, 60, 61, 66, 88, 127, 129, 130, 132, 164, 175, 179, 185, 186]	18
	Media Outlets	[3, 43, 64, 65, 74, 76, 92, 96, 99, 107, 110, 136, 159, 163]	14
3	Personal Blogs	[128, 197, 205]	3
Total			73

well-edited and authored by domain-savvy writers or journalists, placing them within the 2nd tier, due to their moderate outlet control and reasonable credibility. Finally, *Personal Blogs* such as Substack and LinkedIn posts written by independent developers, consultants, or individual practitioners make up 3 of our sources. These sources are placed in the 3rd tier of grey literature due to their informal tone, lack of structured editorial review, and varying levels of author expertise. While they may offer early signals and novel perspectives, we interpret their content with caution during synthesis.

Therefore, our grey literature corpus, as shown in Table 3, includes materials from a mix of businesses and corporations (e.g., Salesforce), academic institutions (e.g., Johns Hopkins), media outlets, and freelance individuals.

3.2.2 Formal Literature. To identify relevant literature, we began by using Google Scholar and DBLP with the same search string applied in our grey literature search. However, the results were found to be noisy, consisting of personal documents hosted on individual websites or repositories. As a result, we decided to use more structured and widely accepted academic databases, following recommendations from established guidelines [111]. Specifically, we searched within the *ACM Digital Library*, *IEEE Xplore*, *Springer Link*, and *Elsevier ScienceDirect*, which are frequently used in systematic and multivocal literature reviews in software engineering. The number of retrieved papers from each source is shown in Table 4. In total, we retrieved 551 papers. To

Table 4. Number of Retrieved Papers From Each Source.

Source	Number of Retrieved Papers
IEEE Xplore Digital Library	17
ACM Digital Library	258
Springer Link Online Library	182
Elsevier ScienceDirect	94
Total	551

determine which papers to include in our study, we applied a predefined set of inclusion and exclusion criteria (detailed in Table 5), developed following SEGRESS [111] for transparent and reproducible review processes. All included papers must be peer-reviewed publications from conferences, workshops, or journals, be related to software engineering, and explicitly discuss technical debt in the context of LLM-generated or AI-assisted code. To avoid ambiguity and instability in the findings of academic papers, we restrict inclusion to peer-reviewed publications because results in preprints may change substantially over time.

Table 5. Inclusion and Exclusion Criteria for Formal Literature.

Inclusion Criteria	
IC1	The paper is peer-reviewed and published in a recognized conference, journal, or workshop
IC2	The paper must be available in one of the online libraries mentioned in Section 3.2.2.
IC3	The paper explicitly discusses technical debt in the context of LLMs and AI-generated or AI-assisted code.
Exclusion Criteria	
EC1	The paper is not written in English, the primary language used for technical and academic research in software engineering.
EC2	The paper focuses on the functional quality or correctness of LLM-generated code but does not directly assess or discuss technical debt, and it is not the main focus of the paper.
EC3	The paper focuses exclusively on the use of LLMs for detecting or mitigating existing technical debt, without examining how LLM-generated outputs introduce, amplify, or propagate technical debt in software artifacts.
EC4	The paper either serves as an extension of a previously accounted-for conference publication or its extended version has already been included.
EC5	The paper is not a traditional research publication and instead serves as an abstract-only submission, doctoral thesis, book, poster, etc.

The selection process was conducted manually by two authors using our set of criteria. Both authors independently reviewed the titles and abstracts of all 551 retrieved papers, marking each for potential inclusion or exclusion based on its relevance. Disagreements or uncertainties were flagged and discussed between all authors until full consensus was reached, ensuring consistency and rigor throughout the process. This initial screening resulted in 49 potential papers. Next, we conducted a full-text review of these 49 papers, applying the inclusion and exclusion criteria more rigorously. During this step, we also extracted any relevant information necessary for answering our research questions (e.g., discussed types of technical debt, proposed tools or metrics) to make sure the papers fit the goal of our study. After this assessment, both authors agreed on the inclusion of 27 papers and excluded the remaining 22 following several rounds of discussion. The primary reasons for exclusion were

that these papers were not directly related to technical debt in the context of LLMs, or they focused primarily on other aspects, such as the functional quality, correctness, or usability of LLM-generated code without addressing technical debt explicitly. To ensure completeness and mitigate any limitations from our initial database search, we also conducted both forward and backward snowballing on the 27 selected papers. This process involved collecting all the references cited in those papers and all subsequent papers that cited them. Through this, we retrieved 1,737 papers. The two authors both manually filtered these by title, abstract, and full text as needed, leading to the inclusion of 4 additional papers. Therefore, in total, our selection process resulted in 31 papers: 27 from the initial formal search and 4 from snowballing.

Table 6. Formal Literature Sources by Venue.

Type	Venue	Sources	Count
Journal	Transactions on Software Engineering and Methodology	[24, 202]	2
	Journal of Systems and Software	[109, 139]	2
	Information and Software Technology	[6]	1
	Computing Surveys	[116]	1
Conference	International Conference on Software Engineering	[30, 147, 200, 204]	4
	International Conference on the Foundations of Software Engineering	[146, 156, 196]	3
	International Conference on Mining Software Repositories	[37, 90]	2
	International Symposium on Empirical Software Engineering and Measurement	[144, 171]	2
	International Conference on Evaluation and Assessment in Software Engineering	[14, 157]	2
	Brazilian Symposium on Software Quality	[172, 177]	2
	International Conference on AI Engineering	[59, 77]	2
	International Conference on Automated Software Engineering	[208]	1
	International Conference on Software Analysis, Evolution and Reengineering	[145]	1
	International Conference on Technical Debt	[9]	1
	International Conference on AI Foundation Models and Software Engineering	[108]	1
	Symposium on Applied Computing	[13]	1
	International Conference on Software Architecture Companion	[150]	1
	Workshop on Machine Learning and Systems	[134]	1
	Workshop on Language Models and Programming Languages	[137]	1
Total			31

As shown in Table 6, the included formal sources span a diverse set of top-tier journals, conferences, and workshops in software engineering. The journal publications come from venues such as Transactions on Software Engineering and Methodology, Empirical Software Engineering, and Journal of Systems and Software, and on the conference side, most papers appear in leading venues like FSE, ICSE, ASE, MSR, and ESEM, along with other more specialized venues such as the International Conference on Technical Debt and Workshop on Language Models and Programming Languages.

3.3 Data Extraction and Synthesis

To answer our study’s research questions, we started by conducting a data extraction process. Specifically, we defined a set of data extraction forms containing targeted questions to help identify key elements for answering each research question. Following the guidelines [83, 111], we collaboratively designed these forms through brainstorming and discussion. For all sources, two authors independently applied the extraction forms to all selected sources. Each author extracted data separately to ensure consistency and completeness. A third author subsequently reviewed the results and resolved any discrepancies or conflicts across the extracted data. For each source, we recorded general metadata (e.g., title, authors, year, source type [grey or formal], and URL), along with study-specific fields. These included:

- The type(s) of technical debt discussed
- Any technical debt concerns unique to LLMs
- Proposed strategies or guidelines for mitigating technical debt introduced by LLMs
- Tools mentioned for measuring or detecting technical debt in LLM-generated code
- Benchmarks or datasets used to evaluate such debt
- Any new metrics or dimensions introduced beyond existing metrics (e.g., SonarQube rules)
- Additional notes and observations

Among the extracted fields listed above, while the first two fields required subjective judgment and interpretation, the remaining fields primarily involved identifying and extracting relevant factual information. For formal sources, we also followed the recommendations of Kitchenham et al. [112] for systematic reviews by computing Cohen’s Kappa coefficient to assess inter-rater reliability and reduce the risks of bias assessment. The inter-rater agreement was strong, with a Cohen’s Kappa of 0.93 for types of technical debt discussed in sources and a range of 0.90–1.00 for the rest of the questions. Any discrepancies and conflicts were resolved to finalize the data extraction of all of our sources. As advised by Garousi et al. [82], explicit traceability links between the extracted data and primary sources are all available in our annotation sheets in the replication package.

In addition, to further ensure clarity in our data extraction, we classified all sources based on how explicitly they frame technical debt. Specifically, we organize our sources into two categories: *direct technical debt evidence*, and *supporting technical debt evidence*. A source is categorized as **direct TD evidence** if it (1) explicitly uses the term technical debt or its categories (e.g., code, design, documentation) in its main title, and (2) frames its main discussion, including findings, risks, or recommendations in terms of technical debt. A source is categorized as **supporting TD evidence** if it (1) discusses phenomena commonly associated with technical debt (e.g., code smells, maintainability issues, quality degradation) in its title, and (2) establishes an explicit connection to technical debt concepts or indicates potential debt accumulation, even if technical debt is not the primary framing. This classification was applied consistently across all sources and is used to guide our interpretation of results. The full categorization of sources is presented in Table 7.

In the data synthesis phase, we organized and interpreted all the extracted data from these sources to derive meaningful insights from both formal and grey literature. The first two authors collaboratively conducted a qualitative analysis, synthesizing findings and drafting the initial results. We used an approach combining deductive coding guided by the existing taxonomy of technical debt from Alves et al. [16] and inductive coding, where new categories were developed iteratively based on recurring patterns in the sources [112]. For example, deductive coding was applied when classifying mentions of technical debt types (e.g., architectural debt, documentation debt), while inductive coding was used to capture emerging concepts unique to the usage of LLMs. All synthesized results were then reviewed and discussed by all authors during multiple meetings to ensure accuracy, completeness, and consensus.

To facilitate a comparative analysis, we present the findings from formal literature, which represent the academic perspective, and those from grey literature, which reflect the practitioner perspective. This separation

Table 7. Classification of Formal and Grey Sources Based on Technical Debt Framing.

Evi- dence	Criteria	Lit.	Titles and Links of Sources
Direct TD Evidence	(1) The source explicitly uses the term TD or a recognized category (e.g., code, design) in its title. (2) The source frames its main discussion, including findings, risks, or recommendations in terms of TD.	Grey	Vibe coding into a tsunami of TD [143], AI makes TD more expensive [88], How AI generated code accelerates TD [164], Generative AI: get ready to multiply your TD [104], The Hidden Debt of AI-Generated Code [115], AI generated code compounds TD [66], Will AI increase TD [179], Navigating AI and TD [166], AI assistants TD generator [29], AI bloating TD [74], Rise of AI TD [65], Hidden TD in LLM apps [175], TD nightmare [159], AI as new source of TD [39], Why AI assistants create TD [34], AI-generated code TD price tag [97], GenAI code debt prediction [23], AI code TD report [98], TD risks [162], Tackling TD with AI [91], Pitfalls of GenAI TD [3], LLMs and TD trap [92], Vibe coding and TD [48], AI-generated code messing with TD [81], Impact of AI-generated code on TD [60], AI-generated code hurts your exit [41], TD in AI-generated code [86], Old-new AI TD risks [129], TD in LLMs [132], TD of LLMs [130], Dark side of AI prototyping: TD [43], Self-healing code to reduce TD [110], AI role in reducing debt [160], How AI can eliminate tech debt [19], Overcoming Ethical debt of AI [76], Using LLMs to pay TD [7], What is AI TD [142], Reducing TD with AI tools [20], Copilot reducing TD [173], Can AI solve the rising costs of TD? [12], Preventing TD [27], Preventing TD from GenAI code [188], AI-resistant TD [148], Dangers of AI Security code debt [5], Moral hazards of TD in LLMs [67], Reduce cost and tech debt of AI code [45], Vibe coding without enterprise TD [107], Balancing benefits of AI with TD [127], CIOs navigating GenAI and debt [64], TD and LLMs frontier [38], Hidden TD in AI [197], Generating Code without Generating TD? [94], How Writing code with AI tools adds TD [128], Managing Agile TD with LLMs [56]
		Formal	Aligning Data Debt with AI-Integrated Software Project [9], 23 shades of self-admitted TD [146], Navigating Challenges and TD in LLMs [134], A Vision for Understanding Ethical Debt in AI-Based Coding [171], Automated Technical Debt Remediation with LLMs [196], PromptDebt: A Comprehensive Study of TD Across LLM Projects [14], The Evolution of TD from DevOps to Generative AI [139]
Support- ing TD Evidence	(1) The source's title discusses phenomena (e.g., code smells, maintainability issues, quality degradation) commonly associated with TD. (2) It discusses signs of TD accumulation in software, and has an explicit connection to TD concepts and its categories.	Grey	AI risks and benefits [174], AI Refactoring tools [61], Productivity vs quality of AI [96], Improve AI code quality [178], AI code review [84], AI code guardrails [52], AI code limitations [99], Challenges of AI code [194], AI Generated code reshaping software [155], Hidden costs of GenAI [136], Copilot quality [205], AI abstraction risks [49], Copilot quality report [163], Vibe then verify AI code [181], AI reshaping architecture [8], AI and iron triangle [106], Functional programming and AI [4], AI and Documentation [185], New generation of AI-augmented coding [186]
		Formal	How AI Increases Short-Term Velocity and Long-Term Complexity in Open-Source Projects [90], A Catalog of Data Smells for Coding Tasks [202], An Study on Architectural Smell Refactoring Using LLMs [150], Are Prompt Engineering and TODO Comments Friends or Foes? [147], Assisting early-stage software with LLMs [6], Analysis of design pattern implementation in LLMs [109], Continuous Quality Improvement of AI-based Systems [144], Safe Automated Refactoring with LLMs [156], Engineering Digital Systems for Humanity [24], Evaluating Source Code Quality with LLMs [177], An Investigation of Test Smells in Code Generation [13], Identification of Redundant Code Using LLMs [59], Increasing Test Coverage using LLMs [172], Challenges for Conversational Agents in SE [116], Do LLMs Provide Links to Code Similar to what they Generate? [37], Fixing Code Smells in Copilot Code using Copilot [208], Categories and detection of suspicious data in AI-based systems [77], A Contamination-Free Code Dataset for Evaluating LLMs [108], Evaluating LLMs in Fixing Maintainability Issues [145], Do Prompt Patterns Affect Code Quality? [157], How Propense Are LLMs at Producing Code Smells? [200], Agent for Code Design Issue Localization [30], Vibe Coding Needs Vibe Reasoning [137], Evaluating Deprecated API Usage in LLMs [204]

allows us to highlight key differences and commonalities in how technical debt related to LLMs is perceived and addressed across academic and industry contexts.

4 Results

4.1 RQ1: What forms of technical debt are introduced by integrating LLM-generated code into software projects?

Table 8 summarizes the identified types of technical debt, along with their definitions and frequencies across the formal and grey literature. The Source column distinguishes between (1) 13 technical debt types drawn from the established taxonomy of Alves et al., which we used as the starting point for answering this research question, and (2) 6 additional debt types that emerged from our multivocal literature review and are not covered by the Alves et al. taxonomy.

Since 2023, there has been a growing awareness of the technical debt risks introduced by incorporating LLMs into software development workflows among practitioners. Figure 4 specifically shows the frequency of mentions in each source type by year. We observe a steady increase in both sources since 2023, reflecting growing concerns about technical debt introduced by LLM-assisted development.

4.1.1 Grey Literature. Across all the 73 sources reviewed, we identified two major categories of concerns: traditional forms of technical debt that were worsened by the use of LLMs, and emerging forms of debt unique to the use of LLMs.

Among the **traditional forms of technical debt** (Alves et al. [16]), the most frequently discussed issue is *code debt* across 56 sources, followed by *design debt* (30 sources) and *documentation debt* (17 sources).

Many industry practitioners highlight that developers often integrate LLM-generated code without fully understanding its logic or verifying its correctness, leading to duplicated or inefficient code structures and increased *code debt*. As noted in Visual Studio Magazine, “*The bottom line is that using Copilot is strongly correlated with mistake code being pushed to the repo*” [163]. This practice increases complexity within the codebase, introduces hidden bugs, and contributes to long-term challenges in maintainability and scalability [98, 130, 162]. As per an article from Tabnine company, “*the code generated for edge cases or intricate business logic often contains errors or simply doesn’t work, requiring the human developer to spend extra time debugging or rewriting it. In effect, the AI can become more of a hindrance than a help on non-trivial coding tasks*” [143]. Developers frequently share concerns about the long-term maintainability of LLM-generated code. Incorrect or illogical code is often difficult to detect and fix, especially when developers lack the expertise to spot flaws or overly trust the model’s output [4]. When these flaws are introduced unchecked, they compound existing code debt, gradually making the system harder to maintain and evolve. Additionally, as noted by Steve Jones, Executive VP of CapGemini, Generative AI generates more code in less time, which increases code complexity and ultimately drives up maintenance costs [104].

Design debt is another concern frequently mentioned in sources that extend these maintainability challenges. Developers emphasize that AI does not understand “your” system [81], meaning that while it can generate code in seconds, such outputs are not necessarily aligned with existing software design decisions and architecture. Without proper oversight, this can lead to code that disregards the overarching system design [115, 129]: “*AI doesn’t understand your system-it just generates code based on statistical patterns. It doesn’t consider your architecture, internal conventions, or existing functionality*” [81].

This potential lack of human oversight and understanding also contributes to a growing codebase where quick AI-generated solutions without any documentation accumulate complexity and *documentation debt* over time, increasing the cost of refactoring and manual developer review. An article in Blar.io talks about how “*developers often hesitate to modify or refactor AI-generated code because they don’t fully grasp its implications*” [38]. Two of the sources also talk about teams losing visibility into how and why certain code decisions were made. For example, the company PortkeyAI included in their blog that “*Ignoring this debt can slow down iteration, hurt user experience, and make scaling painful. New team members struggle to understand the system. Security and*

Table 8. Technical Debt Types with their Definitions and Frequency of Mentions Across Formal and Grey Literature. MLR Denotes Debt Types Identified in this Review.

Source	Type of Debt	Definition	Frequency of Mentions	
			Formal	Grey
Alves et al. [16]	Code Debt	Problems in source code (e.g., poor readability, bad practices) that hinder maintainability.	14	56
	Design Debt	Violations of good design principles (e.g., tightly coupled or overly complex classes).	2	30
	Documentation Debt	Incomplete, outdated, or missing documentation that hinders correct system use or evolution.	0	17
	Infrastructure Debt	Weaknesses in infrastructure that delay upgrades, scaling, or other development activities.	1	12
	Architecture Debt	Issues in system architecture (e.g., modularity, performance, robustness) that require extensive rework.	1	10
	Test Debt	Gaps in testing (missing cases, low coverage, flaky or unrun tests) that reduce confidence in quality.	2	6
	Defect Debt	Known or deferred bugs/defects that accumulate over time, making fixes harder later.	3	1
	Build Debt	Inefficient or poorly defined build processes that slow development and add unnecessary cost.	0	0
	People Debt	Skill gaps, concentrated expertise, or insufficient training that slow progress.	0	0
	Requirement Debt	Partially or poorly implemented requirements that fail to meet quality or performance needs.	0	0
	Service Debt	Suboptimal service or substitution decisions that create long-term maintenance costs.	0	0
	Test Automation Debt	Lack of automated tests that limit continuous integration.	0	0
	Process Debt	Inefficient or outdated processes no longer suited for current needs.	0	0
MLR	Governance Debt	Long-term risks from over-reliance on LLM-generated code, requiring ongoing oversight due to issues like hallucinations and non-determinism.	0	19
	Prompt Debt	Caused by unclear, sensitive, or undocumented prompts that reduce reproducibility and code quality.	8	5
	Fast-integration Debt	Risks from rapidly integrating LLM-generated outputs without proper validation, leading to unstable foundations.	0	13
	Data Debt	Hidden liabilities arising from noisy, low-quality, or opaque data used in training, fine-tuning, or retrieval, which propagate flaws into generated code.	9	0
	Ethical Debt	Accumulated risks of bias, fairness, or accountability failures in LLM-generated code that affect end users.	3	2
	Provenance Debt	Unclear ownership or missing attribution in AI-generated code, creating legal and accountability challenges.	1	1

*compliance risks go unnoticed. And costs grow uncontrollably without visibility” [175]. A developer from Cisco calls this process an *Abstraction Prison*, where software grows every day with AI-generated code, but it is hard to explain how it is working: “The abstraction prison isn’t about the AI being wrong (though it often is). It’s about the growing gap between the code that exists and the code we understand. Every time we accept AI-generated code*

without truly comprehending it, this is like adding another bar to your cell. Before long, you're surrounded by a codebase that runs, passes tests, maybe even ships to production, but you can't explain how or why it works" [49].

Beyond these traditional concerns, 40 sources discuss **five emerging forms of technical debt** in the context of LLM-assisted software development, which we discuss next. Importantly, our focus here is not on technical debt in AI/ML model development, data pipelines, or MLOps infrastructure. Rather, we focus on debt that arises when LLM-generated artifacts, such as code, prompts, and related outputs, are integrated into software development and must later be understood, reviewed, and maintained.

Governance debt (19 sources) is the most frequent LLM-assisted software development debt discussed in grey literature. In our review, this debt refers specifically to the long-term oversight burden created when LLM-generated code or logic is incorporated into software systems, rather than governance challenges in training or operating AI/ML models themselves. This includes the ongoing need for oversight due to issues such as hallucinations and model non-determinism, both of which can undermine reliability and increase the cost of sustaining AI-assisted systems over time, especially in cases where code semantically appears correct but is logically flawed [38, 205]. Based on data collected by International Data Corporation (IDC), 70% of developers using GenAI tools need to remediate as much as 40% of that generated code [49].

Another commonly reported concern is *fast-integration debt* (13 sources), where developers implement AI suggestions quickly, or *vibe code*, without understanding their implications, leading to unstable software foundations [143]. This form of debt is specific to the integration of LLM outputs into software projects, where the speed of adoption can outpace developers' ability to evaluate downstream maintainability and design consequences. A developer describes this in a personal experience where they had to race to finish the product using AI, which resulted in bugs surfacing after weeks of implementation: *"I learned this the hard way during a sprint where we were racing to add features. We had an AI assistant churning out some React components, API handlers...Our velocity metrics looked incredible...Then came the bug reports. Not immediately, that would've been too easy, right? No, these surfaced weeks later. Edge cases AI hadn't considered" [49].* Therefore, this fast integration, in turn, will contribute to the need for more governance in the future.

Prompt debt in 5 sources is another recurring concern that is particularly salient in LLM-assisted development. It refers to unclear or undocumented prompts that degrade code quality and hinder reproducibility, as well as the constant need to determine the proper contextual information in which to add prompts. Unlike traditional software artifacts, prompts directly shape generated implementations, yet they are often not preserved, versioned, or maintained with the same rigor as source code. Developers note issues like prompt brittleness, where minor changes lead to drastically different outputs (*"Small inconsistencies drift into large-scale architectural entropy" [97]*), as well as ambiguous instructions and lack of prompt versioning [104, 164].

Ethical debt refers to fairness and bias concerns that arise when LLM-generated technical debt disproportionately affects end users. Unlike traditional technical debt, ethical debt is often invisible to developers yet deeply consequential for affected users. While the term is increasingly discussed in the context of LLM-assisted development, ethical debt itself is not new; it has long been associated with AI-based systems, particularly in relation to algorithmic fairness, prediction bias, and accountability [40]. What distinguishes the LLM setting in our review is that these risks can be introduced indirectly through generated code, configurations, or design suggestions that developers incorporate into software projects, even when they are not directly building AI/ML models or pipelines, and that such risks may be amplified in this context. The huge scale of LLM training data, which is often unknown, combined with the lack of transparency of their mostly black-box architectures, makes biases and ethical risks integrated into software harder to detect or mitigate. As a result, issues of fairness and accountability are not only inherited but often magnified, creating a larger and less transparent burden for developers and stakeholders. Johns Hopkins' article mentions this debt as *"It's when you're more interested in putting out a solution quickly, which is often the business driver-to be first to market. But you haven't yet looked at*

your solution from an ethical perspective and you end up having a situation where there are biases or other problems within the data or AI solution that haven't been addressed" [76].

An additional form of technical debt discussed in the context of LLM-assisted software development in one source is *provenance (ownership) debt*, where the responsibility and ownership for LLM-generated logic becomes unclear as code cloning becomes more common with the use of AI tools [66]. Here, the concern is not ownership of models or datasets, but ownership and traceability of the generated software artifacts themselves.

4.1.2 Formal Literature. A small but growing body of formal literature examines how LLMs can contribute to technical debt. Across these studies, the most frequently discussed concerns relate to code-level and architectural issues, including code quality and maintainability risks that signal the accumulation of technical debt in software.

14 studies discuss *code debt* and quality issues with software in the era of LLM and AI-assistant usage. One of these studies examines the causal effect of adopting Cursor, a widely used LLM agent assistant, on development velocity and software code debt. He et al. [90] conduct a difference-in-differences design comparing Cursor-adopting GitHub projects with a matched control group of similar GitHub projects that do not use Cursor, finding that the adoption of the AI agent leads to a statistically significant large increase in project-level development velocity, along with a substantial and persistent increase in complexity and code debt. Specifically, static analysis warnings increase by 30% and code complexity increases by 41% post-adoption, all of which are symptoms of code debt. Wang et al. [204] examine seven LLMs, 145 API mappings from eight popular Python libraries, and 28,125 completion prompts. They find that all evaluated LLMs face issues with deprecated API usages inside generated code due to the absence of API deprecation knowledge during model inference, leading to the accumulation of debt when integrated into the software.

One of the most frequently discussed debts (in 9 studies) is *data debt*. Because this category overlaps with prior technical debt work on AI/ML systems, we use it here in a narrower sense. In our review, data debt does not refer broadly to debt in model training pipelines or MLOps infrastructure. Rather, it refers to data-related conditions that shape LLM-generated outputs and later create maintenance burdens when those outputs are incorporated into software projects. This debt stems from the quality of data that influences LLM behavior, including training corpora, fine-tuning datasets, and retrieved contextual data, leading to downstream effects on code generation. While model training is typically performed by LLM providers, data debt is not limited to them. An increasing number of organizations deploy domain-specific or smaller language models [33], apply lightweight fine-tuning, or rely on retrieval-augmented pipelines where the quality of data remains critical. Therefore, the consequences of data debt also manifest at the point of use, directly affecting organizations that integrate LLM-generated code into their software systems. Velasco et al. [200] introduce the Propensity Smelly Code (PSC) metric to estimate the likelihood that code generated by LLMs contains smells. The accumulation of code smells is a direct symptom of a debt that developers have to pay in future, and the authors argue that these smells often stem from the fact that LLMs are trained on publicly available codebases, which themselves may contain poor design patterns or bad practices, and therefore, can be reproduced in the generated outputs of LLMs. Similarly, Vitale et al. [202] characterize the presence of "noise tokens", such as file paths, non-ASCII characters, or personally identifiable information, in LLM training data as a major signal to data debt since such tokens may distract models from learning more meaningful abstractions.

Prompt debt is another type of debt discussed in the formal literature. Aljohani et al. [14] describe it as debt resulting from poorly structured, inefficient, or suboptimal inputs to the LLM systems, and they find it to be the primary source of LLM-specific debt. They also find that instruction-based prompts and few-shot prompts are particularly vulnerable due to their dependence on instruction clarity and example quality, revealing the two prompt techniques that attract the most debt.

Another area receiving attention is *defect debt* discussed in 3 sources. These studies mention self-admitted technical debt (SATD) as an important signal of defects in LLM code. Prior research has shown that SATDs

often indicate underlying technical debt and may co-occur with different types of it such as defect debt in software [31, 158]. Building on this perspective, a study by O’Brien et al. [147] evaluates how Copilot, an LLM coding tool, handles comments containing SATDs. The authors generate over 1,000 function bodies from 380 different TODO-style prompts and find that LLMs often replicate SATD verbatim, including placeholders like “TODO: check divide by zero”. They suggest that prompt sensitivity is a real risk: vague or incomplete instructions can lead to the blind reproduction of debt in generated code. This risk is compounded by the fact that many LLMs are trained on public codebases where such SATDs already exist, potentially reinforcing and propagating poor practices learned from the training data itself. In a related study, Zhang et al. [208] examine Copilot-generated Python code and detect eight distinct types of code smells, further demonstrating early indicators of technical debt that LLMs not only inherit from their training data but also produce outputs that exhibit similar technical debt patterns.

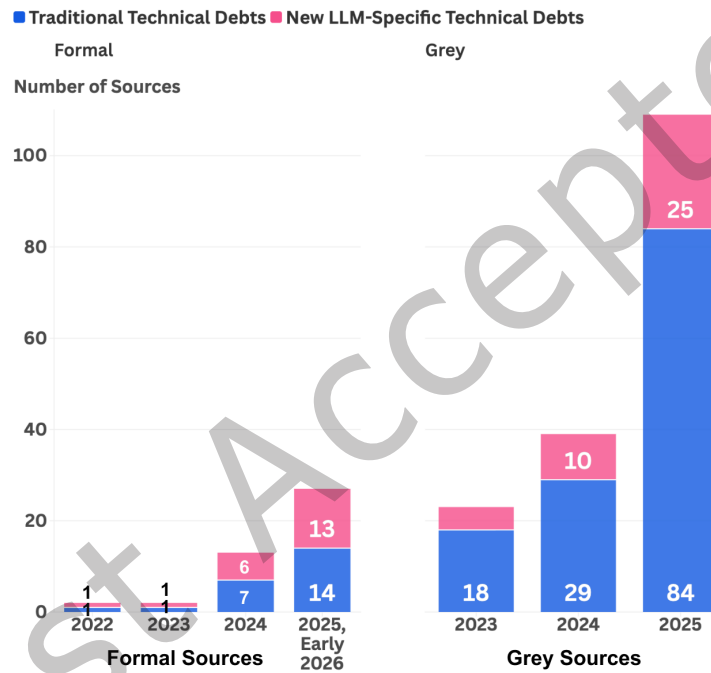


Fig. 4. The Number of Grey and Formal Sources Addressing Traditional and New Technical Debts of LLMs per Year.

Finally, two of the sources introduce discussions around emerging forms of legal and ethical debts in the specific context of integrating LLM-generated artifacts into software projects. Bifulco et al. [37] examine whether LLM-generated code includes links or citations to its sources and conclude that such tools often lack clear attribution, resulting in what they term *provenance debt*. They raise important questions around code ownership and licensing, although this topic remains underexplored compared to more traditional forms of debt. This concern is not about model ownership or dataset governance in general, but about the traceability, attribution, and licensing concerns of generated code that becomes part of a larger software system. Salah et al. [171] describe *ethical debt* as the accumulation of unresolved ethical risks in AI-assisted software development, and emphasize the importance of its sources, manifestations, and outcomes. They specifically highlight mapping accountability

across developers, AI systems, and organizations as an important factor that needs more attention in future research.

LLMs introduce technical debt in two ways: (1) *Traditional debts*, especially *Code*, *Design*, and *Documentation debts*, worsened by hallucinations, weak testing, and limited oversight. (2) *New debts* unique to LLMs: grey literature stresses *Prompt*, *Fast-integration*, *Provenance*, *Ethical*, and *Governance debt*; Formal literature highlights *Data*, *Prompt*, *Provenance*, and *Ethical* debts.

4.2 RQ2: What strategies or guidelines exist to mitigate technical debt introduced by LLMs?

4.2.1 Grey Literature. Strategies offered by grey literature to mitigate technical debt can be broadly grouped into two themes: (1) Tooling and human-in-the-loop frameworks, and (2) Prompt engineering.

A consistent recommendation across 58 out of 73 sources is to follow a human-in-the-loop model. Developers and teams are encouraged to treat LLMs as junior developers, using their outputs as drafts that require review and refinement. An article from Cisco defines this relationship between AI tools and developers more clearly: “Treat AI-generated code like you’re conducting a real code review for, say, a junior developer who’s really enthusiastic but occasionally overconfident. Because that’s essentially what it is” [49], and an article from O’Reilly emphasizes this more by mentioning the need to “trust, but verify”: “Trust the AI to give you a starting point, but verify that the design supports change, testability, and clarity” [148].

Other practices mentioned include applying clean code principles to LLM-generated outputs [66], using unit tests to validate AI-generated suggestions [205], and enforcing multi-layered review processes for cross-functional oversight [38, 163]. Reddit users express similar views, with developers emphasizing the need to review, test, and document the logic behind AI-generated code [164]. Many teams are also integrating tools and establishing organizational practices to monitor and enforce quality in AI-assisted development. Tools such as SonarQube are commonly used for automatic code quality checks, vulnerability detection, and compliance enforcement [174, 181]. Others integrate static analysis tools into AI-augmented review pipelines to reduce duplication and improve maintainability [115]. Additionally, teams use agentic LLM tools like Cursor and HopeAI to flag issues related to maintainability, security, and code standards [48, 186]. While these tools do not directly measure technical debt but they help identify early indicators and potential risks that may contribute to debt accumulation over time. At an organizational level, sources highlight the importance of ongoing developer training, documenting AI involvement in codebases, and prioritizing long-term maintainability over short-term productivity gains [127, 159].

Another set of strategies in 4 sources focuses on improving prompt quality and defining clear boundaries for AI usage. Developers recommend maintaining prompt templates to improve reproducibility and consistency [143]. Others suggest using structured prompts like “act as an expert developer” or providing explicit style/quality constraints to guide the model toward more maintainable outputs [94]. Several sources also emphasize the importance of LLM usage segmentation, clearly delineating when to use fully generative AI or manual coding, to maintain control and ensure consistent quality [38, 104].

The remaining articles, i.e., 11 out of 73 grey literature sources, do not discuss strategies or guidelines to mitigate technical debt introduced by LLMs.

4.2.2 Formal Literature. Strategies in formal literature are largely aligned with those offered by practitioners. These strategies can be grouped into three main categories: (1) Data quality and standards alignment, (2) Prompt engineering, and (3) Tooling and human-in-the-loop frameworks.

A common recommendation across 8 studies involves improving the quality of training and input datasets before integrating them into development pipelines. These works emphasize rigorous preprocessing steps such as labeling, deduplication, and filtering [9, 13, 108, 202], to reduce the risk of accumulation of downstream code

smells and poor-quality generations that will inevitably cause technical debts to pay for. Some papers further advocate aligning technical debt management with industry standards. Akgul et al. [9] propose incorporating ISO/IEC 5338/12207 practices to address data debt, suggesting that aligning with established software lifecycle standards may help mitigate long-term risks and promote sustainable AI-assisted development.

Prompt design and contextual richness also emerged as critical factors contributing to or alleviating various forms of debt in 7 sources. Poorly constructed inputs to the LLMs and a lack of sufficient context in prompts are repeatedly cited as primary causes of code and architectural debt in LLM-generated outputs [14], and research has explored how effective prompt engineering could reduce such debt. Zhang et al. [208] demonstrate that while Copilot-generated Python code often exhibits common smells that signal risks of technical debts, the use of well-structured and specific prompts was able to mitigate up to 87.1% of these issues. Additionally, combining better prompts with test-driven development to systematically prevent technical debt accumulation is another strategy explored in this domain [150].

Beyond inputs and data preparation, 10 sources in the literature emphasize that LLMs are more effective when integrated into broader tool-chains or used within reflective workflows. Rather than relying on one-shot code generation, researchers recommend using LLMs alongside post-generation quality checks or prompting refinement loops. Retrieval-Augmented Generation (RAG) [150] and LLM-as-Judge frameworks [24, 109, 156, 208] are among the most cited strategies in this domain. These methods introduce systematic self-reflection or multi-stage review into the generation process to surface and reduce debt. Nunes et al. [145] underscore the need for post-generation oversight. Their study finds that while LLMs can improve code readability and address certain maintainability concerns, they often introduce new issues or fail to resolve existing ones fully. The authors advocate for fine-tuning and maintaining human oversight throughout the development lifecycle. Mitchell et al. [137] propose Vibe Reasoning, a theoretical system throughout the vibe coding process, to auto-formalize specifications, validate against targets, deliver actionable feedback to the LLM, and allow intuitive developer influence on specifications. Their proof of concept conclusions align with a broader consensus in the literature: effective code generation with LLMs requires high-quality data, carefully designed prompts, continuous validation, and strong human-AI collaboration.

The remaining articles, i.e., 6 out of 31 formal literature sources, do not discuss strategies or guidelines to mitigate technical debt introduced by LLMs.

Top strategies suggested by both grey and formal literature for mitigating technical debt include detailed prompt design, use of code quality tools such as SonarQube, and adding a human-in-the-loop approach to refine and review LLM-generated code before deployment. Academic sources also emphasize improving training data quality to ensure more reliable outputs.

4.3 RQ3: What tools or techniques currently exist to help practitioners detect or measure technical debt in LLM-generated code?

4.3.1 Grey Literature. While only a handful of tools are specifically designed to detect signs and indicators of technical debt introduced by LLM-generated code, several sources highlight a mix of general-purpose, AI-specific, and emerging tools currently in use by practitioners. These can be grouped into: (1) general-purpose static analysis tools, and (2) emerging LLM-aware tools.

The most frequently mentioned tool across 4 sources is SonarQube, which remains a go-to for detecting early signals of technical debt such as code smells, and duplication, even though it is not explicitly tailored to LLM-generated outputs. Other commonly used tools include linters such as ESLint and Pylint, which help flag security vulnerabilities, code smells, and performance bottlenecks that can cause technical debt in software. However, like SonarQube, these tools are designed for general use and not optimized for the nuances of AI-generated

code [128, 181]. CAST Software is mentioned in 2 sources, but again in the context of traditional technical debt detection rather than LLM-specific insights [45, 166].

A number of newer tools are mentioned for their emerging capabilities to analyze or support LLM-assisted development. Amazon CodeWhisperer is noted for its features to check LLM-generated recommendations for security issues in code [61, 96]. GitHub Copilot, while primarily a code generation tool, is gradually introducing responsible AI features, such as flagging insecure or biased outputs and including reference citations [86, 173], [135]. For example, it uses built-in protections to block harmful content and detect prompt injections, which are attempts to manipulate the tool into producing unsafe output. CRken and CodeScene are highlighted as tools for automatically reviewing merge requests and flagging “quick fixes” that may introduce debt over time [20, 52]. Agents such as Swimm, HopeAI, and Graphite are being used to enhance documentation and explainability to avoid technical debt in the future [20, 61, 84, 96]. Snyk, DeepSource, ArmorCode, and Codacy are cited as helpful for detecting early indicators of technical debt such as code smells, dependency issues, and maintainability risks in LLM-assisted workflows [23, 61]. Tabnine is mentioned for its support for model fine-tuning in enterprise environments, aiming to reduce inconsistency and maintain alignment with internal coding standards [143].

4.3.2 Formal Literature. Three papers in formal literature discuss a handful of tools or techniques that can identify the indicators of technical debt in LLM-generated code.

QualAI, currently a framework under development, focuses on continuously monitoring and improving the quality of AI-based systems, with an emphasis on detecting and managing debt in ML applications throughout their lifecycles [144]. By collecting relevant data over time, QualAI can recommend targeted changes to mitigate debt and improve overall system quality. Velasco et al. [200] present CodeSmellEval, which combines a dataset of common code smells with a new Propensity Smelly Code (PSC) evaluation specifically for LLM-generated code. Their results show that higher PSC values correlate with a greater tendency to generate code smells that cause future debt in software. Lastly, to mitigate deprecated API usage in LLM-generated code, another symptom of future debt in code, Wang et al. [204] propose ReplaceAPI, an automated approach that detects deprecated API calls in LLM outputs and replaces them with updated alternatives. This approach helps reduce future debt costs and prevents downstream technical debt caused by evolving software ecosystems. These frameworks offer an early method for benchmarking LLM outputs and could inform future strategies to detect and reduce different aspects of technical debt.

Table 9 summarizes the tools referenced in both literatures, outlining their purposes and indicating whether they are open-source or commercial. Besides tools proposed by formal literature and static analysis tools such as ESLint, Pylint, other tools mentioned in the literature are all commercial.

SonarQube remains the most widely used tool for detecting technical debt indicators. Other commercial tools, such as Swimm, Snyk, and Amazon CodeWhisperer, are increasingly leveraged to detect these common risks in LLM-assisted workflows. Research contributions such as QualAI and CodeSmellEval show early efforts to create systematic frameworks and benchmarks tailored to evaluating technical debt in LLM-generated code.

4.4 RQ4: Are there benchmarks or datasets available to evaluate the technical debt or quality of LLM-generated code?

4.4.1 Grey Literature. Our review of grey literature revealed no standardized benchmarks or datasets currently in use for directly evaluating technical debt or its indicators in LLM-generated code. While practitioners frequently acknowledge the importance of continuous evaluation for the reliable integration of LLMs into software development cycles [74, 81], existing benchmarks focus almost exclusively on binary correctness. As one source emphasized, better benchmarks are needed for “preventing AI from becoming a mindless code churn machine” [81]. This highlights a significant gap in the availability and adoption of LLM-specific evaluation resources. There are

Table 9. Tools Referenced in Grey and Formal Literature that Support Managing or Detecting Technical Debt in LLM-generated code.

Tools	Purpose	Open-source / Commercial
SonarQube [182], ESLint [73], Pylint [161]	Static analysis tools to detect code smells, vulnerabilities, and performance issues	Mostly open-source (includes community editions)
Amazon Code-Whisperer [54], Copilot [55], Tabnine [192]	AI-powered coding assistants that generate code but also provide additional features such as vulnerability scanning, code quality suggestions, and in the case of Copilot, early efforts toward providing references of sources it uses	Commercial
Swimm [190], HopeAI [93], Graphite [85]	AI-powered documentation and code understanding capabilities to improve software maintainability	Commercial
CRken [58], Code-Science [51]	Code review automation that merges requests, flags “quick fixes” likely to add debt	Commercial
Codacy [50], Snyk [180], CAST Software [44]	AI code analysis SaaS platforms to enforce quality and security standards across the code	Commercial
DeepSource [62], ArmorCode [22]	AI DevSecOps platform to assess code quality and SAST	Commercial
CodeSmellEval [200]	Evaluates propensity of smelly code in LLMs	Open-source
QualAI [144]	Continuously monitoring and improving the quality of AI-based systems, to manage debt in ML applications	Open-source (Under Development)

no publicly shared or industry-standard benchmarks tailored to assessing maintainability, long-term quality, or the broader impact of incorporating LLM-generated code into software systems.

4.4.2 Formal Literature. Same as grey sources, formal literature also did not reveal any standardized benchmarks or datasets directly addressing technical debt. There is a clear gap in standardized benchmarks/datasets for technical debt in LLM-generated code. Most studies either adapt existing code smell benchmarks such as CodeXGLUE [125] or CodeSmellData [200] to study narrow, code-smell-related issues or rely on older datasets originally developed for assessing technical debt in software prior to the rise of LLMs [202]. Importantly, this limitation reflects a broader challenge in technical debt research: even for human-written code, widely accepted and comprehensive technical debt benchmarks remain scarce [18, 25, 119]. Existing efforts are often limited in scope and tend to focus on specific debt types or proxy measures rather than long-term debt assessment. Among formal literature, sources emphasize a growing need for better benchmarks and mention the lack of them for current LLM models [202].

No standardized benchmarks exist in either grey or formal literature, though there is growing recognition of the need for LLM-specific benchmarks.

4.5 RQ5: Are there existing technical debt metrics that sufficiently capture the debt introduced by LLMs? If not, what new metrics or dimensions are being suggested?

4.5.1 Grey Literature. Although no concrete new metrics are widely adopted, several sources mention the need for new evaluation dimensions and proposed early-stage alternatives for capturing the unique debt introduced by LLM-generated code. Existing technical debt metrics, such as SonarQube rules and conventional code-quality measures, are referenced across industry discussions and are widely regarded as inadequate for capturing AI-specific debt. In multiple grey literature sources, authors noted that current metrics can surface syntax errors and some maintainability issues; they fail to capture semantic accuracy, architectural fit, and adaptability [38, 186, 205].

Several sources speculate on new dimensions that could better represent LLM-induced debt. A post on LeadDev calls for context-sensitive metrics, focusing on understandability, adaptability, and reviewability [66], mentioning that *“Although AI excels at generating one-off code, its context window is limited. Humans still play a critical role in seeing the bigger picture and understanding the full software portfolio”*. The New Stack proposes tracking post-fix behavior as a signal of long-term debt accumulation not captured by static analysis [110]. Portkey suggests monitoring prompt reliability, token cost sprawl, and integration fragility [175], metrics that are novel and specific to LLM use. Other industry voices that discussed new issues, such as ethical debt, stress that these represent new concerns that call for their own dedicated metrics and evaluation approaches [76, 115, 143]. While SonarQube and similar tools continue to provide partial visibility, the industry recognizes that new, LLM-specific metrics are needed to capture dimensions of debt that static analysis cannot.

4.5.2 Formal Literature. Only a few sources mention any new metrics for detecting or mitigating early signs of technical debt. We shortlisted five papers that either identified shortcomings in current metrics or developed their own metrics.

Pandini et al. [150] combine prompt engineering and retrieval-augmented generation with Arcan, a tool for detecting architectural debt. Their evaluation shows that while such techniques improved results for smaller code debt cases, they were less effective for more complex issues, highlighting limitations of both current tooling and prompt-based mitigation.

Simoes et al. [177] do not strictly focus on LLM code, but do a comparison between SonarQube and LLMs for evaluating the overall quality of code. They found that LLMs are more capable of finding finer details of quality rather than the A to E categorization used in Sonar, which shows the limitation of static tools overall. The results showed that SonarQube rates code quality based on the correcting cost of bad practices, while the LLM seems to rate the code based on the ease of reading. They also showed that there are characteristics captured exclusively by either the LLMs or SonarQube. Similarly, two other studies employ SonarQube rules to assess code quality of LLM-generated code, though they extend its use in different ways. One study complements SonarQube’s automated assessments with human evaluators, noting that judgments of quality can still be subjective [145]. The other also relies on SonarQube but emphasizes its limitations, motivating the need for more context-sensitive metrics that can better capture code quality across varying development scenarios [157].

To find signs of code debt, such as deprecated API usages in LLMs, API Usage Plausibility (AUP) and Deprecated Usage Rate (DUR) are two metrics discussed by Wang et al. [204]. AUP measures how effectively LLMs predict accurate APIs without considering their deprecation status, and DUR calculates the rate of plausible completions that were annotated as deprecated. They argue that a high AUP and low DUR indicate that models predict both up-to-date and accurate APIs.

Lastly, Foidl et al. [77] focus on data debt, introducing two novel metrics: Data Smell Strength and Data Smell Density. These metrics were evaluated using rule-based and ML-based detection techniques across hundreds of datasets. This work demonstrates how new debt types, such as data quality issues, can be measured in a scalable way.

Grey literature mentions the need for new metrics covering dimensions such as context-sensitivity, adaptability, and reviewability. Formal literature acknowledges the limitations of static analysis tools while also exploring specific implementations such as Arcan and extensions of SonarQube, and introducing metrics like Data Smell Strength and Deprecated Usage Rate.

5 Actionable Insights and Future Research Directions

5.1 RQ1: LLMs introduce both traditional and new forms of debt

Our results indicate a growing concern that LLMs are amplifying existing traditional technical debt and its signs in software, and introducing new types of debt, such as prompt, fast integration, and ethical debt. Both practitioners and researchers are beginning to realize that code generation with LLM systems presents a new set of challenges. These challenges not only arise from complex code but also involve how it is produced through prompts and the quality of training data. The rapid pace and scale of integrating LLM-generated code into software systems are also raising pressing questions about how such outputs should be governed and sustained over time.

Future Research Directions:

- (1) **Expanding taxonomies of technical debt:** Taxonomies of debt must evolve beyond traditional categories to capture LLM-specific risks. Existing debt taxonomies (e.g., Alves et al. [16]) are insufficient for capturing the nuanced risks of LLMs. Researchers need to formally study new categories such as prompt debt and governance debt, distinguish them from traditional code or design debt, and show how these interact. For example, our findings show that fast-integration practices can trigger cascading governance risks if left unmanaged. This gap becomes even more pressing in the context of *Software Engineering 3.0* [89], a vision of LLM-native development characterized by conversation-oriented programming where natural language prompts serve as the primary interface between humans and AI teammates. To make this vision sustainable, the community must systematically map the unique forms of debt emerging in this new paradigm and design corresponding strategies for their detection and mitigation.
- (2) **Conducting longitudinal studies:** We currently lack empirical evidence of how LLM-related technical debt accumulates and evolves over time, a gap that is particularly critical given the very nature of debt as something that emerges gradually. Since AI has been increasingly integrated into software development workflows since 2021 [1, 2], projects now routinely leverage LLMs for a wide range of tasks. This makes it an opportune time to conduct longitudinal analyses to determine whether early productivity gains are ultimately offset by growing maintenance and refactoring costs. Such evidence would be invaluable for understanding the long-term sustainability and quality implications of AI-assisted software development.
- (3) **Analyzing socio-technical impacts of AI adoption in software development:** Availability of LLM-based tools is reshaping software development dynamics as these tools increasingly contribute directly to production code [71, 120]. We identify *governance debt* as a pressing concern, highlighting the need for continuous oversight of AI-generated code to ensure quality, and despite the growing adoption of AI within software teams, studies indicate that full reliance on LLM outputs remains risky [87, 167]. Beyond code, we observe emerging challenges in *knowledge management* in developer teams. Prompts and interaction patterns increasingly function as development artifacts, yet are often undocumented or scattered across tools and repositories [131]. This “prompt sprawl” risks loss of system knowledge, weakens accountability, and complicates collaboration [131]. Practices such as prompt registries, versioned documentation, and review workflows for LLM inputs are therefore essential to preserving organizational knowledge [122, 131]. These findings highlight the need for

research that moves beyond purely technical concerns to examine how AI-assisted development reshapes socio-technical aspects in software teams.

📖 Actionable Insights for Practitioners:

- (1) **Safeguards over speed:** Developers must not treat LLM code as “free productivity”. Industry voices emphasize that code quality is often sacrificed for speed. Introducing safeguards such as provenance review (where did the code come from?), governance checks, and systematic prompt documentation can reduce long-term risks.
- (2) **Organizational responsibility:** Beyond individual developers, organizations should institute policies that balance speed and sustainability. For instance, they could require developer review of all LLM-generated pull requests and test coverage before integration into the main codebase.

5.2 RQ2: Strategies for addressing LLM-induced debts are unevenly distributed across debt types

Figure 5 shows that code debt is the primary focus of both grey and formal literature, attracting the most concrete strategies for mitigation. Other forms of debt, particularly new forms such as ethical, provenance, and governance debt, are acknowledged but remain disconnected from actionable practices. The strongest cluster of activity is around tooling and human-in-the-loop frameworks, which benefit from a range of commercial and open-source tools, showing the community’s reliance on oversight mechanisms to tackle debt. By contrast, prompt engineering is tied to a few tools, highlighting the unresolved challenge of standardizing or automating what constitutes an effective prompt with the right amount of context. Data quality and standards alignment are similarly underrepresented, with only a handful of emerging frameworks attempting to fill the gap.

📖 Future Research Directions:

- (1) **Systematically characterize and measure prompt debt:** Prompting has become the central interface between developers and LLMs, and it is emerging as both a source and a solution to technical debt. Poorly designed or brittle prompts can introduce prompt debt [15], while informal practices such as vibe coding contribute to unstable or fast-integration debt, and therefore, more governance debt in the future. In contrast, carefully crafted and well-documented prompts are often described as a key strategy for producing higher-quality, more reliable code [69, 70]. Grey literature highlights this tension most directly, warning that undocumented or inconsistent prompts erode reproducibility and maintainability. Formal studies reinforce the same duality: prompting, especially when combined with techniques like retrieval-augmented generation (RAG), can mitigate design and architectural debt, yet its effectiveness diminishes in large and complex systems. This suggests that while prompting is indispensable, it remains a fragile lever for debt management that needs clearer guidelines, better tooling, and systematic evaluation.
- (2) **Investigate methods for detecting data debt:** Data debt is a silent but central factor for code generation. Formal studies show that when training datasets or retrieved data in RAG systems contain noise, such as stray tokens, file paths, or non-ASCII characters, models inherit and propagate these flaws, creating downstream debt before any code is even written. This problem is compounded by the lack of transparency around commercial LLM training data, where developers often have no visibility into its quality or provenance. Recent work has begun to introduce metrics like Data Smell Density and Data Smell Strength to quantify such issues, but these remain early efforts. Researchers must develop rigorous methods for detecting, quantifying, and mitigating data debt to ensure reproducibility and model trustworthiness.
- (3) **Grounding ethical and provenance debts in empirical studies:** Ethical and provenance debts, stemming from fairness issues in LLM-assisted fixes or unclear code ownership, require stronger empirical grounding [114]. Future work should investigate how opacity in data provenance and

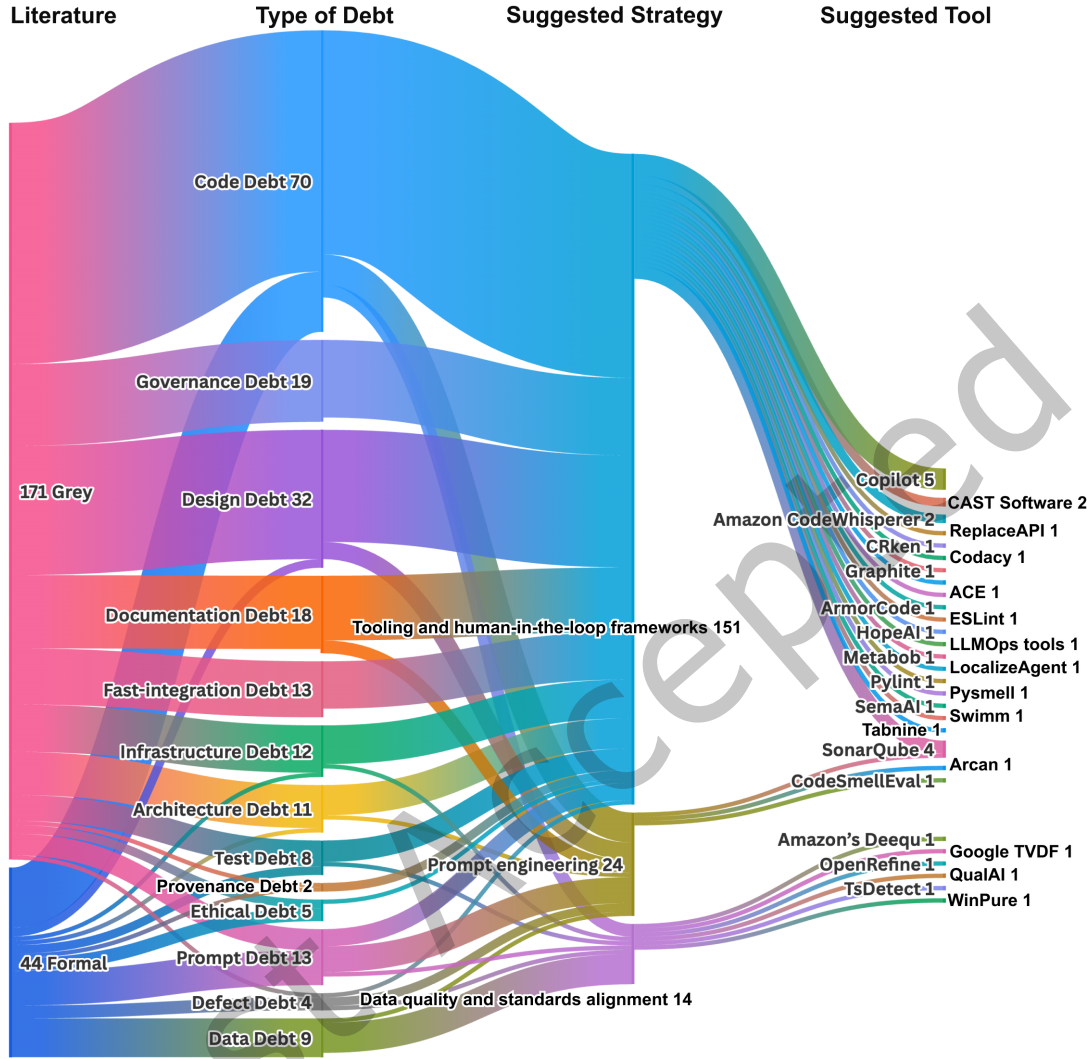


Fig. 5. Mapping Technical Debt Types to Mitigation Strategies Across Grey and Formal Literature.

training sources introduces ethical, legal, and governance liabilities, and how these debts propagate through developer workflows.

✎ Actionable Insights for Practitioners:

- (1) **Human-in-the-loop review systems:** Teams should integrate LLM-generated suggestions into established quality assurance and debt management workflows, incorporating peer review, refactoring, and regression testing to ensure reliability and maintainability.
- (2) **Practical prompt discipline:** Practitioners should begin treating prompts as part of the development lifecycle. Versioning prompts, documenting intent, and testing prompt robustness across variations could prevent prompt debt from silently accumulating.

5.3 RQ3: Tools to detect LLM-induced technical debt are scarce and limited in scope

Industry predominantly relies on static analysis tools such as SonarQube to measure traditional signs of technical debt in software; however, these tools are not designed to accurately capture LLM-induced technical debt. Commercial tools such as CodeWhisperer, CRken, Swimm, and Snyk are beginning to support explainability, documentation, and maintainability in LLM-assisted workflows; however, they are typically offered under proprietary licensing models and are rarely subject to independent or rigorous external evaluation. In contrast, academic work is only just beginning to address this gap: efforts such as QualAI, aimed at monitoring AI-generated systems, and CodeSmellEval, which benchmarks LLM outputs for code smells, mark the first steps toward building more systematic and transparent metrics for LLM-driven debt.

Future Research Directions:

- (1) **Developing open tooling:** Researchers must prioritize the development of transparent, open-source tools designed specifically for LLM-generated code, rather than retrofitting existing static analysis platforms. Traditional static analysis tools are essential for ensuring code quality, but they fail to directly capture LLM-specific risks such as vague prompts, provenance ambiguity (e.g., unclear or missing source attribution), or non-deterministic outputs. New tools should aim to (1) track and version prompts alongside generated code, (2) measure reproducibility and reliability of LLM-generated code (e.g., assessing semantic correctness of code [176]), (3) flag potential provenance or licensing concerns of the generated output, and (4) integrate interpretability and explainability features (e.g., helping developers understand why a certain piece of code was generated [68]). Building these tools openly would allow reproducibility and wider adoption.
- (2) **Establishing rigorous and reproducible evaluation:** Tools should be evaluated using reproducible benchmarks, enabling consistent comparisons across projects and models. Without this, tools risk remaining vendor marketing claims rather than evidence-backed solutions.

Actionable Insights for Practitioners:

- (1) **Reducing vendor dependence:** Commercial tools provide partial coverage and require paid licenses. Teams should combine them with in-house practices for accountability, such as internal dashboards tracking LLM usage or extending existing quality pipelines with LLM-specific checks.
- (2) **Avoiding tool lock-in:** Over-reliance on vendor solutions can limit flexibility and hinder long-term sustainability. Practitioners should push for hybrid approaches (e.g., combining Snyk with open-source refactoring tools) to maintain adaptability.

5.4 RQ4 & RQ5: Metrics and benchmarks to measure LLM-induced technical debt are missing

Currently, no standardized benchmarks or mature metrics exist for directly evaluating technical debt in LLM-generated code. Both grey and formal literature stress that while correctness-based benchmarks are common, they capture only short-term accuracy and overlook deeper concerns such as maintainability, adaptability, and semantic soundness. This gap prevents meaningful comparison across tools and models, leaving developers without reliable ways to assess long-term risks. Establishing LLM-specific benchmarks is therefore a critical future direction, not only to enable fair evaluation but also to guide the design of tools and practices that can genuinely mitigate LLM-induced technical debt.

Future Research Directions:

- (1) **Designing new metrics:** Researchers should go beyond correctness and design metrics that capture semantic accuracy, maintainability, adaptability, and even social dimensions like provenance and ethical implications. New metrics could address the *social and ethical dimensions* of LLM use, such as provenance (can we trace the origin of generated snippets to licensed or unlicensed code?), reproducibility of outputs (does the same prompt yield consistent results across runs?), and fairness/ethical

risk (does the generated code propagate harmful stereotypes or security vulnerabilities) [169, 170]. Developing these metrics in open-source frameworks would allow integration into CI/CD pipelines, ensuring they become actionable rather than just theoretical constructs.

- (2) **Developing community-driven benchmarks:** Shared, public benchmarks for LLM-generated code are essential. Similar to how CodeXGLUE [125] advanced code intelligence tasks, or SWE-Bench [103] made the evaluation of LLMs easier for bug fixing, an equivalent suite would drive comparability in technical debt research.

📖 Actionable Insights for Practitioners:

- (1) **Adopt conservative validation:** Until robust benchmarks exist, practitioners must treat LLM integration as experimental. Manual review, regression testing, and stress testing should be mandatory.
- (2) **Collect organizational evidence:** Teams can build their own internal benchmarks by tracking local data (e.g., bug rates, maintenance costs of LLM code). This internal evidence base can guide decisions until standardized metrics emerge.

6 Threats to Validity

Construct Validity: While our source selection process involved manual filtering and is susceptible to human error, we mitigated this by having two authors independently review all sources and resolve disagreements through discussion. We also followed a rigorous quality assessment process for both formal and grey sources. For grey literature, we reviewed full content; for formal papers, we examined titles, abstracts, and full texts as needed. Another potential threat lies in the formulation of search strings. To address this, we tested multiple variations and selected the one that yielded the most relevant results with minimal noise. To reduce bias in data extraction and coding, two authors independently coded all data. For subjective coding dimensions, we calculated Cohen's Kappa coefficients to assess inter-rater reliability, achieving high agreement scores (above 0.9). All extractions were reviewed in joint sessions with all authors to ensure consistency and accuracy. Study selection bias may have occurred during screening, as inclusion and exclusion decisions involve researcher judgment. To mitigate this risk, we defined explicit inclusion criteria, conducted independent screening, and resolved disagreements through discussion.

Internal Validity: Our search results may have been influenced by the internal algorithms of the digital libraries and search engines used, which are not publicly documented. To mitigate this risk, we used four major academic databases (IEEE Xplore Digital Library, ACM Digital Library, Elsevier ScienceDirect, and SpringerLink Online Library) and searched grey sources through the Google search engine, increasing coverage and reducing dependence on a single indexing mechanism. Additionally, grey literature varies in methodological rigor compared to peer-reviewed studies. We mitigated this risk by applying a structured quality assessment checklist and excluding sources that did not meet the threshold.

External Validity: We limited our selection to English-language sources accessible through the selected platforms. This may have excluded some relevant material and affected generalizability. In addition, our reliance on the technical debt terms in the search string may bias the sample toward studies and practitioner discussions that explicitly frame quality concerns as those types of debt, while not including related work discussed under alternative terms such as maintainability, software evolution, or code quality. To mitigate this, we used an inclusive search string of 13 established technical debts in the literature, followed by extensive manual filtering to remove irrelevant entries. This allowed us to include as many relevant sources as possible while maintaining quality and focus. We invested significant time in the manual inspection process, which led to the removal of over a thousand irrelevant sources. The included studies span diverse tools, domains, and development contexts. However, findings may not generalize uniformly across all programming ecosystems or organizational settings. In addition, while our source retrieval process followed systematic guidelines and did not favor any specific

source type, a large portion of our grey literature corpus (52%) consists of company blogs and industry reports, which reflects the current state of practitioner discourse in LLM-assisted development that may introduce a bias toward grey literature's perspectives.

7 Conclusion

This study provides a comprehensive examination of the risks and challenges posed by technical debt in the context of LLM-generated code, along with strategies proposed for its detection and mitigation from both practitioner and research perspectives. Through a multivocal literature review of 104 sources, we find that LLMs intersect with existing forms of technical debt, especially code, design, and documentation debts the most. In addition, they introduce new forms of debt, most notably data debt, fast-integration debt, and governance debt that remain underexplored and insufficiently addressed. Given the abundance of discussion around traditional debts of software from LLMs and forms of debts that are new to the special needs of LLMs, our findings emphasize the need for deeper investigation into the long-term implications of LLM use within software engineering workflows. We highlight several strategies repeatedly discussed in both literatures, including tooling and human-in-the-loop frameworks, prompt engineering, and data quality alignment, as central approaches for managing LLM-related debt. We also categorize several commercial and open-source tools suggested for practice, from the integration of LLMs with static analysis tools such as SonarQube and Pylint to manage early indicators of technical debt in software, to emerging LLM-assisted platforms such as Amazon CodeWhisperer, Swimm, and Copilot. However, our findings show that current solutions remain fragmented, heavily commercial in nature, and limited in their ability to fully address the breadth of LLM-related technical debt. Finally, we identify a striking absence of standardized benchmarks and LLM-specific metrics, highlighting a major gap that limits systematic evaluation of LLM-generated code and its long-term maintainability. Addressing this gap alongside the emerging forms of debt we highlight marks an important direction for future research, one that will be critical for ensuring sustainable and responsible integration of LLMs in software development.

In the future, research must move beyond documenting risks toward building actionable solutions. This includes designing standardized benchmarks and LLM-specific metrics, developing robust frameworks for prompt management and evaluation, and creating open, community-driven datasets that reduce reliance on commercial black-box models. Equally important is studying the temporal evolution of LLM-related debt, to understand not only how such debt emerges but also how it accumulates or can be repaid over time. By pursuing these directions, the software engineering community can better align research, practice, and tooling to manage technical debt in the era of LLMs.

References

- [1] [n. d.]. AI Copilot Code Quality: 2025 Data Suggests 4x Growth in Code Clones - GitClear. https://www.gitclear.com/ai_assistant_code_quality_2025_research
- [2] 2025. A Year In, GitHub Measures AI-Based Copilot's Productivity Boost -. <https://visualstudiomagazine.com/Articles/2022/09/13/copilot-impact.aspx>
- [3] Anthony Abdulla. 2024. The Pitfalls of Generative AI: How to Avoid Deepening Technical Debt. <https://www.rtinsights.com/the-pitfalls-of-generative-ai-how-to-avoid-deepening-technical-debt/>
- [4] AdaBeat. 2025. Functional Programming and generative AI. <https://adabeat.com/fp/functional-programming-and-generative-ai/>
- [5] AEI. 2024. The Hidden Danger of Security Code Debt: Why Thorough Vetting Is Crucial Before Deployment. <https://www.aei.org/technology-and-innovation/the-hidden-danger-of-security-code-debt-why-thorough-vetting-is-crucial-before-deployment/>
- [6] Thea Lovise Ahlgren, Helene Fønsteli Sunde, Kai-Kristian Kemell, and Anh Nguyen-Duc. 2025. Assisting early-stage software startups with LLMs: Effective prompt engineering and system instruction design. *Information and Software Technology* 187 (Nov. 2025), 107832. doi:10.1016/j.infsof.2025.107832
- [7] Mohamed Ahmed. 2024. How Software Developers Can Use LLMs To Pay Technical Debt. <https://aifordevelopers.io/how-software-developers-can-use-llms-to-pay-technical-debt/>

- [8] AITInc. 2025. How AI-generated code is reshaping software architecture. <https://ait.inc/tech-stuffs/how-ai-generated-code-is-reshaping-software-architecture/>
- [9] Nilay Yorgancılar Akgül, Tugba Taskaya Temizel, Ozden Ozcan Top, and Pelin Dayan Akman. 2025. Aligning Data Debt with AI-Integrated Software Project Lifecycle Processes: A Standard-Based Mapping Approach. IEEE Computer Society, 1–11. doi:10.1109/TechDebt66644.2025.00008
- [10] Danylo Albuquerque, Everton Guimarães, Graziela Tonin, Pilar Rodríguez, Mirko Perkusich, Hyggo Almeida, Angelo Perkusich, and Ferdinandy Chagas. 2023. Managing Technical Debt Using Intelligent Techniques - A Systematic Mapping Study. *IEEE Trans. Softw. Eng.* 49, 4 (April 2023), 2202–2220. doi:10.1109/TSE.2022.3214764
- [11] Reem Alfayez, Wesam Alwehaibi, Robert Winn, Elaine Venson, and Barry Boehm. 2020. A systematic literature review of technical debt prioritization. In *Proceedings of the 3rd International Conference on Technical Debt* (Seoul, Republic of Korea) (*TechDebt '20*). Association for Computing Machinery, New York, NY, USA, 1–10. doi:10.1145/3387906.3388630
- [12] alixpartners. 2024. Can AI solve the rising costs of technical debt? <https://www.alixpartners.com/insights/102jlar/can-ai-solve-the-rising-costs-of-technical-debt/>
- [13] Ahmed Aljohani and Hyunsook Do. 2024. From Fine-tuning to Output: An Empirical Investigation of Test Smells in Transformer-Based Test Code Generation. In *Proceedings of the 39th ACM/SIGAPP Symposium on Applied Computing (SAC '24)*. Association for Computing Machinery, New York, NY, USA, 1282–1291. doi:10.1145/3605098.3636058
- [14] Ahmed Aljohani and Hyunsook Do. 2025. PromptDebt: A Comprehensive Study of Technical Debt Across LLM Projects. In *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering (EASE '25)*. Association for Computing Machinery, New York, NY, USA, 371–382. doi:10.1145/3756681.3756976
- [15] Ahmed Aljohani and Hyunsook Do. 2025. PromptDebt: A Comprehensive Study of Technical Debt Across LLM Projects. doi:10.1145/3756681.3756976 arXiv:2509.20497 [cs].
- [16] Niccoli S.R. Alves, Leilane F. Ribeiro, Viviane Caires, Thiago S. Mendes, and Rodrigo O. Spínola. 2014. Towards an Ontology of Terms on Technical Debt. In *2014 Sixth International Workshop on Managing Technical Debt*. 1–7. doi:10.1109/MTD.2014.9
- [17] Theodoros Amanatidis, Nikolaos Mittas, Aikaterini Moschou, et al. 2020. Evaluating the agreement among technical debt measurement tools: building an empirical benchmark of technical debt liabilities. *Empirical Software Engineering* 25, 6 (2020), 4161–4204. doi:10.1007/s10664-020-09869-w
- [18] Theodoros Amanatidis, Nikolaos Mittas, Athanasia Moschou, Alexander Chatzigeorgiou, Apostolos Ampatzoglou, and Lefteris Angelis. 2020. Evaluating the agreement among technical debt measurement tools: building an empirical benchmark of technical debt liabilities. *Empirical Software Engineering* 25, 5 (Sept. 2020), 4161–4204. doi:10.1007/s10664-020-09869-w
- [19] Ismail Amla. 2024. How AI eliminates tech debt and unlocks new software possibilities. <https://www.kyndryl.com/us/en/about-us/news/2024/10/how-ai-eliminates-tech-debt-improves-software-development>
- [20] API4AI. 2024. Reducing Technical Debt with AI Code Review Tools. <https://medium.com/@API4AI/reducing-technical-debt-with-ai-code-review-tools-5ec03a3c8d57>
- [21] archdia. [n.d.]. dv8. <http://archdia.com/>
- [22] ArmorCode. 2025. ArmorCode. <https://www.armorcode.com/>
- [23] Armorcode. 2025. Your GenAI Code Debt Is Coming Due. <https://www.armorcode.com/blog/your-genai-code-debt-is-coming-due-heres-what-gartner-predicts>
- [24] Marco Autili, Martina De Sanctis, Paola Inverardi, and Patrizio Pelliccione. 2025. Engineering Digital Systems for Humanity: A Research Roadmap. *ACM Trans. Softw. Eng. Methodol.* 34, 5 (May 2025), 130:1–130:33. doi:10.1145/3712006
- [25] Paris Avgeriou, Ipek Ozkaya, Heiko Koziolek, Zadia Codabux, and Neil Ernst. 2025. Manifesto from Dagstuhl Perspectives Workshop 24452 – Reframing Technical Debt. arXiv:2505.13009 [cs.SE] <https://arxiv.org/abs/2505.13009>
- [26] Paris C. Avgeriou, Davide Taibi, Apostolos Ampatzoglou, Francesca Arcelli Fontana, Terese Besker, Alexander Chatzigeorgiou, Valentina Lenarduzzi, Antonio Martini, Athanasia Moschou, Ilaria Pigazzini, Nyyti Saarimäki, Darius Daniel Sas, Saulo Soares de Toledo, and Angeliki Agathi Tsintzira. 2021. An Overview and Comparison of Technical Debt Measurement Tools. *IEEE Software* 38, 3 (2021), 61–71. doi:10.1109/MS.2020.3024958
- [27] Axel. 2023. Preventing Technical Debt with AI Code Reviews. <https://metabob.com/blog-articles/preventing-technical-debt-with-ai-code-reviews.html>
- [28] Nasreen Azad and Sami Hyrynsalmi. 2024. Multivocal Literature Review on DevOps Critical Success Factors. In *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering* (Salerno, Italy) (*EASE '24*). Association for Computing Machinery, New York, NY, USA, 520–527. doi:10.1145/3661167.3661236
- [29] Sudheer Bandaru. 2025. AI Coding Assistants: Superpower or Technical Debt Generator? <https://www.hivel.ai/blog/ai-coding-assistants-superpower-or-technical-debt-generator>
- [30] Fraol Batole, David OBrien, Tien N. Nguyen, Robert Dyer, and Hriday Rajan. 2025. An LLM-Based Agent-Oriented Approach for Automated Code Design Issue Localization. IEEE Computer Society, 1320–1332. doi:10.1109/ICSE55347.2025.00100

- [31] Gabriele Bavota and Barbara Russo. 2016. A large-scale empirical study on self-admitted technical debt. In *Proceedings of the 13th International Conference on Mining Software Repositories (Austin, Texas) (MSR '16)*. Association for Computing Machinery, New York, NY, USA, 315–326. doi:10.1145/2901739.2901742
- [32] Woubshet Nema Behutiye, Pilar Rodríguez, Markku Oivo, and Ayşe Tosun. 2017. Analyzing the concept of technical debt in the context of agile software development: A systematic literature review. *Information and Software Technology* 82 (Feb. 2017), 139–158. doi:10.1016/j.infsof.2016.10.004
- [33] Peter Belcak, Greg Heinrich, Shizhe Diao, Yonggan Fu, Xin Dong, Saurav Muralidharan, Yingyan Celine Lin, and Pavlo Molchanov. 2025. Small Language Models are the Future of Agentic AI. arXiv:2506.02153 [cs.AI] <https://arxiv.org/abs/2506.02153>
- [34] Rocío Belfiore. 2025. Write Code Like It's 2010: Why AI Assistants Create Technical Debt. <https://www.bairesdev.com/blog/why-ai-assistants-create-technical-debt/>
- [35] Terese Besker, Antonio Martini, and Jan Bosch. 2018. Managing architectural technical debt: A unified model and systematic literature review. *Journal of Systems and Software* 135 (2018), 1–16. doi:10.1016/j.jss.2017.09.025
- [36] João Paulo Biazotto, Daniel Feitosa, Paris Avgeriou, and Elisa Yumi Nakagawa. 2024. Technical debt management automation: State of the art and future perspectives. *Information and Software Technology* 167 (2024), 107375. doi:10.1016/j.infsof.2023.107375
- [37] Daniele Bifulco, Pietro Cassieri, Giuseppe Scanniello, Massimiliano Di Penta, and Fiorella Zampetti. 2025. Do LLMs Provide Links to Code Similar to what they Generate? A Study with Gemini and Bing CoPilot. doi:10.48550/arXiv.2501.12134 arXiv:2501.12134 [cs].
- [38] Blar.io. 2025. Technical debt part 2: LLMs, AI and the new Frontier. <https://blar.io/blog/technical-debt-part-2-llms-ai-and-the-new-frontier>
- [39] David Blatherwick. 2023. AI Generated Code - Short Term Wins, Long Term Losses. <https://akfpartners.com/growth-blog/ai-is-it-the-new-source-of-tech-debt>
- [40] Justus Bogner, Roberto Verdecchia, and Ilias Gerostathopoulos. 2021. Characterizing Technical Debt and Antipatterns in AI-Based Systems: A Systematic Mapping Study. In *2021 IEEE/ACM International Conference on Technical Debt (TechDebt)*. 64–73. doi:10.1109/TechDebt52882.2021.00016
- [41] BootstrappedFounder. 2025. Why AI-Generated Code Hurts Your Exit. <https://thebootstrappedfounder.com/why-ai-generated-code-hurts-your-exit/>
- [42] Jenna Butler, Jina Suh, Sankeerti Haniyur, and Constance Hadley. 2025. Dear Diary: A Randomized Controlled Trial of Generative AI Coding Tools in the Workplace. In *2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. 319–329. doi:10.1109/ICSE-SEIP66354.2025.00034
- [43] Elena Calvillo. 2025. The Dark Side Of AI Prototyping: Technical Debt. https://promptledproduct.substack.com/p/the-dark-side-of-ai-prototyping-technical?utm_source=publication-search
- [44] castsoftware. 2025. castsoftware. <https://www.castsoftware.com/highlight>
- [45] Castsoftware. 2025. Ongoing reduction of maintenance costs & technical debt. <https://learn.castsoftware.com/reduce-cost-and-tech-debt>
- [46] Humberto Cervantes and Rick Kazman. 2020. Software archinaut: a tool to understand architecture, identify technical debt hotspots and manage evolution. In *Proceedings of the 3rd International Conference on Technical Debt (Seoul, Republic of Korea) (TechDebt '20)*. Association for Computing Machinery, New York, NY, USA, 115–119. doi:10.1145/3387906.3388633
- [47] checkstyle. [n. d.]. checkstyle. <https://checkstyle.org/>
- [48] Mike Chen. 2025. Vibe Coding: The Future of AI-Powered Development or a Recipe for Technical Debt? <https://blog.bitsrc.io/vibe-coding-the-future-of-ai-powered-development-or-a-recipe-for-technical-debt-2fd3a0a4e8b3>
- [49] Cisco. 2025. Escaping Abstraction: Why AI-Generated Code Demands More Than Just Trust. <https://learningnetwork.cisco.com/s/blogs/a0DKd00006bjy76MAA/escaping-abstraction-why-ai-generated-code-demands-more-than-just-trust>
- [50] codacy. 2025. codacy. <https://www.codacy.com/>
- [51] CodeScene. 2025. CodeScene. <https://codescene.com/>
- [52] Codescene. 2025. Automated quality gates for AI coding. <https://codescene.com/use-cases/ai-code-quality>
- [53] CodeScene Inc. 2004. CodeScene. <https://codescene.io>.
- [54] CodeWhisperer. 2025. CodeWhisperer. <https://workshops.aws/categories/CodeWhisperer>
- [55] Copilot. 2025. Copilot. <https://www.microsoft.com/en-us/microsoft-copilot/for-individuals?form=MY02P9>
- [56] Enrique De Coss. 2021. Agile Technical Debt: A Practical Guide to Management. <https://muuktest.com/blog/technical-debt-in-agile>
- [57] John Crickett. [n. d.]. LinkedIn. https://www.linkedin.com/posts/johnrickett_beware-llms-have-enabled-the-rapid-creation-activity-7291443929529352193-JjMw/
- [58] crken. 2025. crken. <https://api4.ai/crken>
- [59] Shamse Tasnim Cynthia. 2025. Identification and Optimization of Redundant Code Using Large Language Models. *IEEE Computer Society*, 261–263. doi:10.1109/CAIN66642.2025.00042
- [60] Antoine Dauplain. 2025. The Impact of AI-Generated Code on Technical Debt and the Need for Accelerated Codebase Appraisal. <https://cerfacs.fr/coop/hpcsoftware-codemetrics-kpis>

- [61] DavidW. 2024. 15 AI Code Refactoring Tools You Should Know. <https://overcast.blog/15-ai-code-refactoring-tools-you-should-know-50cf38d26877>
- [62] deepsource. 2025. deepsource. <https://deepsources.com/>
- [63] designite. [n. d.]. designite. <https://www.designite-tools.com/>
- [64] Bill Doerrfeld. 2024. How CIOs navigate generative AI in the enterprise. <https://www.cio.com/article/1313757/how-cios-navigate-generative-ai-in-the-enterprise.html>
- [65] Bill Doerrfeld. 2024. Will the Rise of Generative AI Increase Technical Debt? <https://devops.com/will-the-rise-of-generative-ai-increase-technical-debt/>
- [66] Bill Doerrfeld. 2025. How AI-generated code compounds technical debt. <https://leaddev.com/software-quality/how-ai-generated-code-accelerates-technical-debt>
- [67] DukeUniversity. 2024. The Moral Hazards of Technical Debt in Large Language Models: Why Moving Fast and Breaking Things Is Bad. <https://read.dukeupress.edu/critical-ai/article-abstract/doi/10.1215/2834703X-11205182/390857/The-Moral-Hazards-of-Technical-Debt-in-Large>
- [68] Ramtin Ehsani, Esteban Parra, Sonia Haiduc, and Preetha Chatterjee. 2025. Hierarchical Knowledge Injection for Improving LLM-based Program Repair. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 1440–1452. doi:10.1109/ASE63991.2025.00122
- [69] Ramtin Ehsani, Sakshi Pathak, and Preetha Chatterjee. 2025. Towards Detecting Prompt Knowledge Gaps for Improved LLM-guided Issue Resolution. In *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*. 699–711. doi:10.1109/MSR66628.2025.00107
- [70] Ramtin Ehsani, Sakshi Pathak, Esteban Parra, Sonia Haiduc, and Preetha Chatterjee. 2025. What characteristics make ChatGPT effective for software issue resolution? An empirical study of task, project, and conversational signals in GitHub issues. *Empirical Software Engineering* 31, 1 (Nov. 2025). doi:10.1007/s10664-025-10745-8
- [71] Ramtin Ehsani, Sakshi Pathak, Shriya Rawal, Abdullah Al Mujahid, Mia Mohammad Imran, and Preetha Chatterjee. 2026. Where Do AI Coding Agents Fail? An Empirical Study of Failed Agentic Pull Requests in GitHub. In *23rd International Conference on Mining Software Repositories (MSR '26)*, April 13–14, 2026, Rio de Janeiro, Brazil. <https://arxiv.org/abs/2601.15195>
- [72] Neil A. Ernst, Stephany Bellomo, Ipek Ozkaya, Robert L. Nord, and Ian Gorton. 2015. Measure it? Manage it? Ignore it? software practitioners and technical debt. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering (Bergamo, Italy) (ESEC/FSE 2015)*. Association for Computing Machinery, New York, NY, USA, 50–60. doi:10.1145/2786805.2786848
- [73] ESLint. 2025. ESLint. <https://eslint.org/>
- [74] Paul Estes. 2025. Is AI Bloating Your Technical Debt? What You Need to Know. <https://www.virtasant.com/ai-today/is-ai-bloating-your-technical-debt-what-you-need-to-know>
- [75] Eduardo Fernandes, Johnatan Oliveira, Gustavo Vale, Thanis Paiva, and Eduardo Figueiredo. 2016. A review-based comparative study of bad smell detection tools. In *Proceedings of the 20th International Conference on Evaluation and Assessment in Software Engineering (Limerick, Ireland) (EASE '16)*. Association for Computing Machinery, New York, NY, USA, Article 18, 12 pages. doi:10.1145/2915970.2915984
- [76] Mike Field. 2024. Overcoming 'Ethical Debt'. <https://engineering.jhu.edu/news/overcoming-ethical-debt/>
- [77] Harald Foidl, Michael Felderer, and Rudolf Ramler. 2022. Data Smells: Categories, Causes and Consequences, and Detection of Suspicious Data in AI-based Systems. *IEEE Computer Society*, 229–239. doi:10.1145/3522664.3528590
- [78] Francesca Arcelli Fontana, Pietro Braione, and Marco Zanoni. 2012. Automatic detection of bad smells in code: An experimental assessment. *Journal of Object Technology* 11, 2 (Aug. 2012), 5:1–38. doi:10.5381/jot.2012.11.2.a5
- [79] Francesca Arcelli Fontana, Ilaria Pigazzini, and Riccardo Roveda abd Marco Zanoni. 2016. Automatic Detection of Instability Architectural Smells. In *2016 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE Computer Society.
- [80] Francesca Arcelli Fontana, Ilaria Pigazzini, Riccardo Roveda, Damian Tamburri, Marco Zanoni, and Elisabetta Di Nitto. 2017. Arcan: A Tool for Architectural Smells Detection. In *2017 IEEE International Conference on Software Architecture Workshops (ICSAW)*. 282–285. doi:10.1109/ICSAW.2017.16
- [81] Edvaldo Freitas. 2025. How AI-Generated Code is messing with your Technical Debt. <https://kodus.io/en/ai-generated-code-is-messing-with-your-technical-debt/>
- [82] Vahid Garousi, Michael Felderer, and Mika V. Mäntylä. 2019. Guidelines for including grey literature and conducting multivocal literature reviews in software engineering. *Information and Software Technology* 106 (2019), 101–121. doi:10.1016/j.infsof.2018.09.006
- [83] Vahid Garousi and Barış Küçük. 2018. Smells in software test code: A survey of knowledge in industry and academia. *Journal of Systems and Software* 138 (2018), 52–81. doi:10.1016/j.jss.2017.12.013
- [84] Graphite. 2025. AI is writing code—here's why it also needs to review that code. <https://graphite.com/blog/ai-code-review-for-ai-generated-code>
- [85] GraphiteAI. 2025. GraphiteAI. <https://graphite.com/>

- [86] GrowthAcceleration. 2025. What Is Technical Debt in AI Codes & How to Manage It. <https://www.growthaccelerationpartners.com/blog/what-is-technical-debt-in-ai-generated-codes-how-to-manage-it>
- [87] Everton Guimaraes and Nathalia Nascimento. 2025. AI in the Software Development Lifecycle: Insights and Open Research Questions. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering* (Clarion Hotel Trondheim, Trondheim, Norway) (FSE Companion '25). Association for Computing Machinery, New York, NY, USA, 1353–1357. doi:10.1145/3696630.3730538
- [88] HackerNews. 2024. AI makes tech debt more expensive. <https://news.ycombinator.com/item?id=42137527>
- [89] Ahmed E. Hassan, Gustavo A. Oliva, Dayi Lin, Boyuan Chen, Zhen Ming, and Jiang. 2024. Towards AI-Native Software Engineering (SE 3.0): A Vision and a Challenge Roadmap. arXiv:2410.06107 [cs.SE] <https://arxiv.org/abs/2410.06107>
- [90] Hao He, Courtney Miller, Shyam Agarwal, Christian Kästner, and Bogdan Vasilescu. 2026. Speed at the Cost of Quality: How Cursor AI Increases Short-Term Velocity and Long-Term Complexity in Open-Source Projects. In *23rd International Conference on Mining Software Repositories (MSR '26)*, April 13–14, 2026, Rio de Janeiro, Brazil.
- [91] Jorge Hernández. [n. d.]. Tackling Technical Debt with Generative AI. <https://www.encora.com/insights/tackling-technical-debt-with-generative-ai>
- [92] Rob Hirschfeld. 2023. Are LLMs Leading DevOps Into a Tech Debt Trap? <https://devops.com/are-llms-leading-devops-into-a-tech-debt-trap/>
- [93] hopeai. 2025. hopeai. <https://hopeai.com/>
- [94] Reka Horvath. 2023. Generating Code without Generating Technical Debt? <https://sourcery.ai/blog/chatgpt-maintainable-code>
- [95] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large Language Models for Software Engineering: A Systematic Literature Review. *ACM Trans. Softw. Eng. Methodol.* 33, 8, Article 220 (Dec. 2024), 79 pages. doi:10.1145/3695988
- [96] Neil C. Hughes. 2025. AI coding assistants promise more productivity, but do they deliver. <https://cybernews.com/ai-news/ai-coding-assistants-productivity-quality-tradeoffs/>
- [97] InclusionCloud. 2025. AI Is Changing How We Code. But Is Technical Debt the Price Tag? <https://inclusioncloud.com/insights/blog/ai-generated-code-technical-debt/>
- [98] InfoQ. 2025. AI-Generated Code Creates New Wave of Technical Debt, Report Finds. <https://www.infoq.com/news/2025/11/ai-code-technical-debt/>
- [99] InfoWorld. 2025. Why AI-generated code isn't good enough (and how it will get better). <https://www.infoworld.com/article/3844363/why-ai-generated-code-isnt-good-enough-and-how-it-will-get-better.html>
- [100] Code Inspector. [n. d.]. Code Inspector. <https://www.code-inspector.com/>
- [101] Mohammad Talal Jamil, Shamsa Abid, and Shafay Shamil. 2025. Can LLMs Generate Higher Quality Code Than Humans? An Empirical Study. In *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*. 478–489. doi:10.1109/MSR66628.2025.00081
- [102] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. 2025. A Survey on Large Language Models for Code Generation. *ACM Trans. Softw. Eng. Methodol.* (July 2025). doi:10.1145/3747588 Just Accepted.
- [103] Carlos E. Jimenez, John Yang, Alexander Wetteg, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? arXiv:2310.06770 [cs.CL] <https://arxiv.org/abs/2310.06770>
- [104] Steve Jones. 2023. Generative AI: get ready to multiply your Technical Debt. <https://blog.metamirror.io/generative-ai-get-ready-to-multiply-your-technical-debt-fd60ee7d7f4e?gi=ce9b36e14d46>
- [105] jsprit. [n. d.]. jsprit. <https://sites.google.com/site/santiagoavidal/projects/jsprit>
- [106] Ted Julian. 2025. AI Generated Code: Revisiting the Iron Triangle in 2025. <https://www.askflux.ai/blog/ai-generated-code-revisiting-the-iron-triangle-in-2025>
- [107] Leslie kanthan. 2025. Can Developers Embrace “Vibe Coding” Without Enterprise Embracing AI Technical Debt? <https://www.unite.ai/can-developers-embrace-vibe-coding-without-enterprise-embracing-ai-technical-debt/>
- [108] Jonathan Katzy, Razvan Mihai Popescu, Arie van Deursen, and Maliheh Izadi. 2025. The Heap: A Contamination-Free Multilingual Code Dataset for Evaluating Large Language Models. In *2025 IEEE/ACM Second International Conference on AI Foundation Models and Software Engineering (Forge)*. 151–155. doi:10.1109/Forge66646.2025.00025
- [109] Dae-Kyoo Kim. 2025. Comparative analysis of design pattern implementation validity in LLM-based code refactoring. *Journal of Systems and Software* 230 (Dec. 2025), 112519. doi:10.1016/j.jss.2025.112519
- [110] Kirimgeray Kirimli. 2024. How to Use Self-Healing Code to Reduce Technical Debt. <https://thenewstack.io/how-to-use-self-healing-code-to-reduce-technical-debt/>
- [111] Barbara Kitchenham, Lech Madeyski, and David Budgen. 2023. SEGREGS: Software Engineering Guidelines for REporting Secondary Studies. *IEEE Transactions on Software Engineering* 49, 3 (2023), 1273–1298. doi:10.1109/TSE.2022.3174092
- [112] Barbara Kitchenham, O. Pearl Brereton, David Budgen, Mark Turner, John Bailey, and Stephen Linkman. 2009. Systematic literature reviews in software engineering – A systematic literature review. *Information and Software Technology* 51, 1 (2009), 7–15. doi:10.1016/j.infsof.2008.09.009 Special Section - Most Cited Articles in 2002 and Regular Research Papers.

- [113] Howard Kleinwaks, Ann Batchelor, and Thomas Bradley. 2023. Technical debt in systems engineering—A systematic literature review. *Systems Engineering* 26 (04 2023). doi:10.1002/sys.21681
- [114] Rrezarta Krasniqi, Depeng Xu, and Marco Vieira. 2025. SE Perspective on LLMs: Biases in Code Generation, Code Interpretability, and Code Security Risks. *ACM Comput. Surv.* 58, 5, Article 137 (Dec. 2025), 16 pages. doi:10.1145/3774324
- [115] Rejith Krishnan. 2025. The Hidden Debt of AI-Generated Code in Agentic AI Systems: A Deep Dive. <https://www.lowtouch.ai/the-hidden-debt-of-ai-generated-code/>
- [116] Stefano Lambiasi, Gemma Catolino, Fabio Palomba, and Filomena Ferrucci. 2024. Motivations, Challenges, Best Practices, and Benefits for Bots and Conversational Agents in Software Engineering: A Multivocal Literature Review. *ACM Comput. Surv.* 57, 4 (Dec. 2024), 93:1–93:37. doi:10.1145/3704806
- [117] Jason Lefever, Yuanfang Cai, Humberto Cervantes, Rick Kazman, and Hongzhou Fang. 2021. On the Lack of Consensus Among Technical Debt Detection Tools. In *2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. 121–130. doi:10.1109/ICSE-SEIP52600.2021.00021
- [118] Valentina Lenarduzzi, Terese Besker, Davide Taibi, Antonio Martini, and Francesca Arcelli Fontana. 2021. A systematic literature review on Technical Debt prioritization: Strategies, processes, factors, and tools. *Journal of Systems and Software* 171 (2021), 110827. doi:10.1016/j.jss.2020.110827
- [119] Valentina Lenarduzzi, Nyyti Saarimäki, and Davide Taibi. 2019. The Technical Debt Dataset. In *Proceedings of the Fifteenth International Conference on Predictive Models and Data Analytics in Software Engineering (Recife, Brazil) (PROMISE'19)*. Association for Computing Machinery, New York, NY, USA, 2–11. doi:10.1145/3345629.3345630
- [120] Hao Li, Haoxiang Zhang, and Ahmed E. Hassan. 2025. The Rise of AI Teammates in Software Engineering (SE) 3.0: How Autonomous Coding Agents Are Reshaping Software Engineering. arXiv:2507.15003 [cs.SE] <https://arxiv.org/abs/2507.15003>
- [121] Zengyang Li, Paris Avgeriou, and Peng Liang. 2015. A systematic mapping study on technical debt and its management. *J. Syst. Softw.* 101, C (March 2015), 193–220. doi:10.1016/j.jss.2014.12.027
- [122] LinkedIn. 2025. Prompts are the new code reviews we're not doing. https://www.linkedin.com/posts/elif-kinli-29b24160_prompts-are-the-new-code-reviews-were-not-share-7425660890416537600-SN3V/?utm_source=share&utm_medium=member_desktop&rcm=ACoAAaAkeAvIBS15DuuN9etGwRe_ny2hLmdkBWSk
- [123] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. In *Proceedings of the 37th International Conference on Neural Information Processing Systems (New Orleans, LA, USA) (NIPS '23)*. Curran Associates Inc., Red Hook, NY, USA, Article 943, 15 pages.
- [124] Yue Liu, Ratnadira Widyasari, Yanjie Zhao, Ivana Clairine Irsan, and David Lo. 2026. Debt Behind the AI Boom: A Large-Scale Empirical Study of AI-Generated Code in the Wild. arXiv:2603.28592 [cs.SE] <https://arxiv.org/abs/2603.28592>
- [125] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin Clement, Dawn Drain, Daxin Jiang, Duyu Tang, Ge Li, Lidong Zhou, Linjun Shou, Long Zhou, Michele Tufano, Ming Gong, Ming Zhou, Nan Duan, Neel Sundaresan, Shao Kun Deng, Shengyu Fu, and Shujie Liu. 2021. CodeXGLUE: A Machine Learning Benchmark Dataset for Code Understanding and Generation. arXiv:2102.04664 [cs.SE]
- [126] Bjorn Arild Lunde and Ricardo Colomo-Palacios. 2020. Continuous practices and technical debt: a systematic literature review. In *2020 20th International Conference on Computational Science and Its Applications (ICCSA)*. IEEE, Cagliari, Italy, 40–44. doi:10.1109/ICCSA50381.2020.00018
- [127] Simon Ma. 2025. Balancing the benefits of AI with technical debt. <https://futurecio.tech/balancing-the-benefits-of-ai-with-technical-debt/>
- [128] Tshepang Mangoejane. 2025. How Writing Code with AI Tools Adds to Technical Debt and How to Manage It. <https://www.linkedin.com/pulse/how-writing-code-ai-tools-adds-technical-debt-tshepang-mangoejane-pwd4f/>
- [129] Medium. 2023. Decoding the ‘(Old) New Technical Debt’: The Risks of Unofficial & AI-Generated IaC. <https://medium.com/@alex.pinheiro.soares/decoding-the-old-new-technical-debt-the-risks-of-unofficial-ai-generated-iac-6230ddabf514>
- [130] Medium. 2024. What Is the Technical Debt of Large Language Models (LLMs) and How Does It Affect Us? <https://sandar-ali.medium.com/what-is-the-technical-debt-of-large-language-models-llms-and-how-does-it-affect-us-484efe4d9552>
- [131] Medium. 2025. The Hidden Crisis of Prompt Sprawl (And How to Fix It). https://medium.com/@martin_rodek/the-hidden-crisis-of-prompt-sprawl-and-how-to-fix-it-9b5e65cd10fc
- [132] Medium. 2025. The Hidden Technical Debt in Large Language Model (LLM) Systems. <https://medium.com/@remotelyintelligent/the-hidden-technical-debt-in-large-language-model-llm-systems-0581e347dbc6>
- [133] Ana Melo, Roberta Fagundes, Valentina Lenarduzzi, and Wylliams Barbosa Santos. 2022. Identification and measurement of Requirements Technical Debt in software development: A systematic literature review. *J. Syst. Softw.* 194, C (Dec. 2022), 21 pages. doi:10.1016/j.jss.2022.111483
- [134] Ahmed Menshawy, Zeeshan Nawaz, and Mahmoud Fahmy. 2024. Navigating Challenges and Technical Debt in Large Language Models Deployment. In *Proceedings of the 4th Workshop on Machine Learning and Systems (EuroMLSys '24)*. Association for Computing Machinery, New York, NY, USA, 192–199. doi:10.1145/3642970.3655840

- [135] Microsoft. 2025. Data, Privacy, and Security for Microsoft 365 Copilot. <https://learn.microsoft.com/en-us/copilot/microsoft-365/microsoft-365-copilot-privacy>
- [136] MIT. 2025. The Hidden Costs of Coding With Generative AI. <https://sloanreview.mit.edu/article/the-hidden-costs-of-coding-with-generative-ai/>
- [137] Jacqueline Mitchell and Yasser Shaaban. 2025. Position: Vibe Coding Needs Vibe Reasoning: Improving Vibe Coding with Formal Verification. In *Proceedings of the 1st ACM SIGPLAN International Workshop on Language Models and Programming Languages* (Singapore, Singapore) (LMPL '25). Association for Computing Machinery, New York, NY, USA, 84–90. doi:10.1145/3759425.3763390
- [138] Ran Mo, Yuanfang Cai, Rick Kazman, Lu Xiao, and Qiong Feng. 2021. Architecture Anti-Patterns: Automatically Detectable Violations of Design Principles. *IEEE Transactions on Software Engineering* 47, 5 (2021), 1008–1028. doi:10.1109/TSE.2019.2910856
- [139] Sergio Moreschini, Elvira-Maria Arvanitou, Elisavet-Persefoni Kanidou, Nikolaos Nikolaidis, Ruoyu Su, Apostolos Ampatzoglou, Alexander Chatzigeorgiou, and Valentina Lenarduzzi. 2026. The Evolution of Technical Debt from DevOps to Generative AI: A multivocal literature review. *Journal of Systems and Software* 231 (2026), 112599. doi:10.1016/j.jss.2025.112599
- [140] Donatien Koulla Moulla, Ernest Mnkandla, Hayatou Oumarou, and Thomas Fehlmann. 2024. Technical Debt Measurement: An Exploratory Literature Review.
- [141] Code MRI. [n. d.]. Code MRI. <https://codemri.com/>
- [142] Tim Mucci. 2025. What is technical debt? <https://www.ibm.com/think/topics/technical-debt>
- [143] Alin Muntean. 2025. How to avoid vibe coding your way into a tsunami of tech debt. <https://www.tabnine.com/blog/how-to-avoid-vibe-coding-your-way-into-a-tsunami-of-tech-debt/>
- [144] Nicole Novielli, Rocco Oliveto, Fabio Palomba, Fabio Calefato, Giuseppe Colavito, Vincenzo De Martino, Antonio Della Porta, Giammaria Giordano, Emanuela Guglielmi, Filippo Lanubile, Luigi Quaranta, Gilberto Recupito, Simone Scalabrino, Angelica Spina, and Antonio Vitale. 2024. Continuous Quality Improvement of AI-based Systems: the QualAI Project. In *Proceedings of the 18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM '24)*. Association for Computing Machinery, New York, NY, USA, 603–607. doi:10.1145/3674805.3695393
- [145] Henrique Nunes, Eduardo Figueiredo, Larissa Rocha, Sarah Nadi, Fischer Ferreira, and Geanderson Esteves. 2025. Evaluating the Effectiveness of LLMs in Fixing Maintainability Issues in Real-World Projects. In *2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. 669–680. doi:10.1109/SANER64311.2025.00069 ISSN: 2640-7574.
- [146] David OBrien, Sumon Biswas, Sayem Imtiaz, Rabe Abdalkareem, Emad Shihab, and Hridesh Rajan. 2022. 23 shades of self-admitted technical debt: an empirical study on machine learning software. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2022)*. Association for Computing Machinery, New York, NY, USA, 734–746. doi:10.1145/3540250.3549088
- [147] David OBrien, Sumon Biswas, Sayem Mohammad Imtiaz, Rabe Abdalkareem, Emad Shihab, and Hridesh Rajan. 2024. Are Prompt Engineering and TODO Comments Friends or Foes? An Evaluation on GitHub Copilot. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE '24)*. Association for Computing Machinery, New York, NY, USA, 1–13. doi:10.1145/3597503.3639176
- [148] O'Reilly. 2025. Building AI-Resistant Technical Debt. <https://www.oreilly.com/radar/building-ai-resistant-technical-debt/>
- [149] Thanis Paiva, Amanda Damasceno, Eduardo Figueiredo, and Cláudio Sant'Anna. 2017. On the evaluation of code smells and detection tools. *Journal of Software Engineering Research and Development* 5, 1 (Oct. 2017), 7. doi:10.1186/s40411-017-0041-1
- [150] Gabriele Pandini, Antonio Martini, Adela Nedisan Videsjorden, and Francesca Arcelli Fontana. 2025. An Exploratory Study on Architectural Smell Refactoring Using Large Languages Models. In *2025 IEEE 22nd International Conference on Software Architecture Companion (ICSA-C)*. 462–471. doi:10.1109/ICSA-C65153.2025.00070 ISSN: 2768-4288.
- [151] Sida Peng, Eirini Kalliamvakou, Peter Cihon, and Mert Demirer. 2023. The Impact of AI on Developer Productivity: Evidence from GitHub Copilot. arXiv:2302.06590 [cs.SE] <https://arxiv.org/abs/2302.06590>
- [152] Judith Perera, Ewan Tempero, Yu-Cheng Tu, and Kelly Blincoe. 2023. Quantifying Technical Debt: A Systematic Mapping Study and a Conceptual Model. arXiv:2303.06535 [cs.SE]
- [153] Judith Perera, Ewan Tempero, Yu-Cheng Tu, and Kelly Blincoe. 2024. A Systematic Mapping Study Exploring Quantification Approaches to Code, Design, and Architecture Technical Debt. *ACM Trans. Softw. Eng. Methodol.* 33, 7, Article 177 (Sept. 2024), 44 pages. doi:10.1145/3675393
- [154] pmd. [n. d.]. pmd. <https://pmd.github.io/>
- [155] Polcode. 2025. AI-Generated Code: Say Hello to Legacy 2.0. <https://polcode.com/resources/blog/ai-generated-code-say-hello-to-legacy-2-0/>
- [156] Dorin Pomian, Abhiram Bellur, Malinda Dilhara, Zarina Kurbatova, Egor Bogomolov, Andrey Sokolov, Timofey Bryksin, and Danny Dig. 2024. EM-Assist: Safe Automated ExtractMethod Refactoring with LLMs. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering (FSE 2024)*. Association for Computing Machinery, New York, NY, USA, 582–586. doi:10.1145/3663529.3663803

- [157] Antonio Della Porta, Stefano Lambiase, and Fabio Palomba. 2025. Do Prompt Patterns Affect Code Quality? A First Empirical Assessment of ChatGPT-Generated Code. doi:10.48550/arXiv.2504.13656 arXiv:2504.13656 [cs].
- [158] Aniket Potdar and Emad Shihab. 2014. An Exploratory Study on Self-Admitted Technical Debt. In *2014 IEEE International Conference on Software Maintenance and Evolution*. 91–100. doi:10.1109/ICSME.2014.31
- [159] Alexander Procter. 2025. Why AI-generated code is creating a technical debt nightmare. <https://www.okoone.com/spark/technology-innovation/why-ai-generated-code-is-creating-a-technical-debt-nightmare/>
- [160] Aravind Putrevu. 2024. Reduce Tech Debt: AI's Role in Efficient Coding. <https://www.coderabbit.ai/blog/reduce-tech-debt-ai-s-role-in-efficient-coding>
- [161] PyLint. 2025. PyLint. <https://www.pylint.org/>
- [162] QodoAI. 2025. Technical Debt and AI: Understanding the Tradeoff and How to Stay Ahead. <https://www.qodo.ai/blog/technical-debt/>
- [163] David Ramel. 2025. New GitHub Copilot Research Finds 'Downward Pressure on Code Quality'. <https://visualstudiomagazine.com/articles/2024/01/25/copilot-research.aspx>
- [164] Reddit. 2024. How AI generated code accelerates technical debt. https://www.reddit.com/r/programming/comments/1it1usc/how_ai_generated_code_accelerates_technical_debt/
- [165] Francisco Ribeiro, Claudio Spiess, Prem Devanbu, and Sarah Nadi. 2026. On LLMs' Internal Representation of Code Correctness. arXiv:2512.07404 [cs.SE] <https://arxiv.org/abs/2512.07404>
- [166] Greg Rivera. 2024. Artificial Intelligence and Technical Debt: Navigating the New Frontier. <https://www.castsoftware.com/pulse/artificial-intelligence-and-technical-debt-navigating-the-new-frontier>
- [167] Sadra Sabouri, Philipp Eibl, Xinyi Zhou, Morteza Ziyadi, Nenad Medvidovic, Lars Lindemann, and Souti Chattopadhyay. 2025. Trust Dynamics in AI-Assisted Development: Definitions, Factors, and Implications. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. 1678–1690. doi:10.1109/ICSE55347.2025.00199
- [168] Hina Saeeda, Muhammad Ovais Ahmad, and Tomas Gustavsson. 2024. A Multivocal Literature Review on Non-Technical Debt in Software Development: An Insight into Process, Social, People, Organizational, and Culture Debt. *e-Informatica Software Engineering Journal* 18, 1 (Sept. 2024), 240101. doi:10.37190/e-Inf240101 Available online: 25 Sep. 2023.
- [169] Amirali Sajadi, Kostadin Damevski, and Preetha Chatterjee. 2025. How Safe Are AI-Generated Patches? A Large-scale Study on Security Risks in LLM and Agentic Automated Program Repair on SWE-bench. arXiv:2507.02976 [cs.CR] <https://arxiv.org/abs/2507.02976>
- [170] Amirali Sajadi, Binh Le, Anh Nguyen, Kostadin Damevski, and Preetha Chatterjee. 2025. Do LLMs consider security? an empirical study on responses to programming questions. *Empirical Software Engineering* 30, 3 (2025), 101.
- [171] Dina Salah. 2025. Invisible Risks, Visible Code: A Vision for Understanding Ethical Debt in AI-Based Coding. In *Proceedings of the 19th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*. 442–446. doi:10.1109/ESEM64174.2025.00074
- [172] Shexmo Richarlison Ribeiro dos Santos, Raiane Eunice S. Fernandes, Marcos Cesar Barbosa dos Santos, Michel S. Soares, Fabio Gomes Rocha, and Sabrina Marczak. 2024. Increasing Test Coverage by Automating BDD Tests in Proofs of Concepts (POCs) using LLM. In *Proceedings of the XXIII Brazilian Symposium on Software Quality (SBQS '24)*. Association for Computing Machinery, New York, NY, USA, 519–525. doi:10.1145/3701625.3701637
- [173] George Schildge. 2023. What is GitHub Copilot: Reducing Technical Debt. <https://matrixmarketinggroup.com/github-copilot-reducing-technical-debt/>
- [174] Legit Security. 2025. AI Code Generation: The Risks and Benefits of AI in Software. <https://www.legitsecurity.com/aspm-knowledge-base/ai-code-generation-benefits-and-risks>
- [175] Drishti Shah. 2025. The hidden technical debt in LLM apps. <https://portkey.ai/blog/the-hidden-technical-debt-in-llm-apps>
- [176] Arindam Sharma and Cristina David. 2025. Assessing Correctness in LLM-Based Code Generation via Uncertainty Estimation. arXiv:2502.11620 [cs.SE] <https://arxiv.org/abs/2502.11620>
- [177] Igor Regis da Silva Simões and Elaine Venson. 2024. Evaluating Source Code Quality with Large Language Models: a comparative study. In *Proceedings of the XXIII Brazilian Symposium on Software Quality (SBQS '24)*. Association for Computing Machinery, New York, NY, USA, 103–113. doi:10.1145/3701625.3701650
- [178] Site24x7. 2025. Improving code quality and reducing technical debt with ChatGPT. <https://www.site24x7.com/learn/how-to-improve-code-quality-using-chatgpt.html>
- [179] Alejandra Sivori. 2024. Will Generative AI Increase Salesforce Technical Debt? <https://www.salesforceben.com/will-generative-ai-increase-salesforce-technical-debt/>
- [180] snyk. 2025. snyk. <https://snyk.io/>
- [181] Sonar. 2025. AI-generated code introduces hidden challenges. <https://www.sonarsource.com/solutions/ai/>
- [182] SonarQube. 2025. SonarQube. <https://www.sonarsource.com/products/sonarqube/>
- [183] Armando Sousa, Lincoln Rocha, and Ricardo Britto. 2023. Architectural Technical Debt - A Systematic Mapping Study. In *Proceedings of the XXXVII Brazilian Symposium on Software Engineering (Campo Grande, Brazil) (SBES '23)*. Association for Computing Machinery, New York, NY, USA, 196–205. doi:10.1145/3613372.3613399
- [184] SQuORE. [n.d.]. SQuORE. <https://codemri.com/>

- [185] Stackoverflow. 2025. Why do developers love clean code but hate writing documentation? <https://stackoverflow.blog/2024/12/19/developers-hate-documentation-ai-generated-toil-work/>
- [186] Ritz Steytler. 2025. LinkedIn Discussion. https://www.linkedin.com/posts/ritzsteytler_how-ai-generated-code-compounds-technical-share-7302615149125795840-XNsr/
- [187] Margaret-Anne Storey. 2026. From Technical Debt to Cognitive and Intent Debt: Rethinking Software Health in the Age of AI. arXiv:2603.22106 [cs.SE] <https://arxiv.org/abs/2603.22106>
- [188] Ritika Strauss. 2024. How Sema prevents technical debt from GenAI code. <https://www.semasoftware.com/blog/stack-overflow-podcast-how-sema-prevents-technical-debt-from-genai-code-2>
- [189] Structure101. [n. d.]. Structure101. <https://structure101.com/>
- [190] Swimm. 2025. Swimm. <https://swimm.io/>
- [191] Symfony. [n. d.]. Symfony. <https://insight.symfony.com/>
- [192] tabnine. 2025. tabnine. <https://www.tabnine.com/>
- [193] Mahan Tafreshipour, Aaron Imani, Eric Huang, Eduardo Santana de Almeida, Thomas Zimmermann, and Iftekhar Ahmed. 2025. Prompting in the Wild: An Empirical Study of Prompt Evolution in Software Repositories. In *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*. 686–698. doi:10.1109/MSR66628.2025.00106
- [194] TimeXtender. 2025. Addressing the Rising Challenges with AI-Generated Code. <https://www.timextender.com/blog/data-empowered-leadership/challenges-with-ai-generated-code>
- [195] Edith Tom, Aybüke Aurum, and Richard Vidgen. 2013. An exploration of technical debt. *J. Syst. Softw.* 86, 6 (June 2013), 1498–1516. doi:10.1016/j.jss.2012.12.052
- [196] Adam Tornhill, Markus Borg, Nadim Hagatulah, and Emma Söderberg. 2025. ACE: Automated Technical Debt Remediation with Validated Large Language Model Refactorings. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering* (Clarion Hotel Trondheim, Trondheim, Norway) (*FSE Companion '25*). Association for Computing Machinery, New York, NY, USA, 1318–1324. doi:10.1145/3696630.3730565
- [197] Tomasz Tunguz. 2025. Hidden Technical Debt in AI. <https://tomtunguz.com/hidden-technical-debt-in-ai/>
- [198] Thomas Valentin, Ardi Madadi, Gaetano Sapia, and Marcel Böhme. 2025. Incoherence as Oracle-less Measure of Error in LLM-Based Code Generation. arXiv:2507.00057 [cs.PL] <https://arxiv.org/abs/2507.00057>
- [199] Bhanu Prakash Vangala, Ali Adibifar, Ashish Gehani, and Tanu Malik. 2026. AI-Generated Code Is Not Reproducible (Yet): An Empirical Study of Dependency Gaps in LLM-Based Coding Agents. arXiv:2512.22387 [cs.SE] <https://arxiv.org/abs/2512.22387>
- [200] Alejandro Velasco, Daniel Rodriguez-Cardenas, Luftar Rahman Alif, David N. Palacio, and Denys Poshyvanyk. 2025. How Propense Are Large Language Models at Producing Code Smells? A Benchmarking Study. In *2025 IEEE/ACM 47th International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER)*. 96–100. doi:10.1109/ICSE-NIER66352.2025.00025 ISSN: 2832-7632.
- [201] Arturo Villa, Jorge Ocharán-Hernández, Juan Carlos Pérez-Arriaga, and Xavier Limón. 2022. A Systematic Mapping Study on Technical Debt in Microservices. 182–191 pages. doi:10.1109/CONISOFT55708.2022.00032
- [202] Antonio Vitale, Rocco Oliveto, and Simone Scalabrino. 2025. A Catalog of Data Smells for Coding Tasks. *ACM Trans. Softw. Eng. Methodol.* 34, 4 (April 2025), 113:1–113:32. doi:10.1145/3707457
- [203] Alexander von Zitzewitz. 2019. Mitigating Technical and Architectural Debt with Sonargraph. In *2019 IEEE/ACM International Conference on Technical Debt (TechDebt)*. 66–67. doi:10.1109/TechDebt.2019.00022
- [204] Chong Wang, Kaifeng Huang, Jian Zhang, Yebo Feng, Lyuye Zhang, Yang Liu, and Xin Peng. 2025. LLMs Meet Library Evolution: Evaluating Deprecated API Usage in LLM-Based Code Completion. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. 885–897. doi:10.1109/ICSE55347.2025.00245
- [205] Jesse Warden. 2025. GitHub Copilot Research Finds “Downward Pressure on Code Quality”. <https://jessewarden.com/2024/01/github-copilot-research-finds-downward-pressure-on-code-quality.html>
- [206] Lu Xiao, Yuanfang Cai, Rick Kazman, Ran Mo, and Qiong Feng. 2016. Identifying and quantifying architectural debt. In *Proceedings of the 38th international conference on software engineering*. 488–498.
- [207] Liang Yu. 2025. Paradigm shift on Coding Productivity Using GenAI. In *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering (EASE '25)*. Association for Computing Machinery, New York, NY, USA, 708–713. doi:10.1145/3756681.3757081
- [208] Beiqi Zhang, Peng Liang, Qiong Feng, Yujia Fu, and Zengyang Li. 2024. Copilot-in-the-Loop: Fixing Code Smells in Copilot-Generated Python Code using Copilot. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE '24)*. Association for Computing Machinery, New York, NY, USA, 2230–2234. doi:10.1145/3691620.3695290

Received 16 October 2025; revised 15 April 2026; accepted 25 May 2026